CHAPTER 10
Working with Editors

IN THIS CHAPTER

Working with the vim editor

Exploring nano

Understanding Emacs

Getting comfortable with KWrite

Looking at Kate

Using the GNOME editor

Before you start your shell scripting career, it's wise to gain proficiency using at least one text editor in Linux. Using features such as searching, cutting, and pasting allows you to develop your shell scripts more quickly.

You have the choice of several editors. Many individuals find a particular editor whose functionality they love and use that one exclusively. This chapter provides a brief sampling of a few of the text editors you can employ in the Linux world.

Visiting the vim Editor

The vi editor was one of the early editors used on Unix systems. It uses the console graphics mode to emulate a text-editing window, allowing you to see the lines of your file; move around within the file; and insert, edit, and replace text.

Although it is quite possibly the most complicated editor in the world (at least in the opinion of those who don't like it), vi provides many features that have made it a staple for programmers and system administrators for decades.

When the GNU Project ported the vi editor to the open source world, they chose to make some improvements to it. Because it extended the original vi editor found in the Unix world, the developers also renamed it "vi improved" or *vim*.

This section walks you through the basics of using the vim editor to edit your text shell script files.

Checking your vim package

Before you begin your exploration of the vim editor, it's a good idea to understand what vim package your Linux system has installed. On some distributions, you will have the full vim package installed and an alias for the `vi` command, as shown on this CentOS distribution:

```
$ alias vi
alias vi='vim'
$
$ which vim
/usr/bin/vim
$
$ ls -l /usr/bin/vim
-rwxr-xr-x. 1 root root 3522560 Nov 11 14:08 /usr/bin/vim
$
```

Notice that the program file's long listing does not show any linked files (see Chapter 3, "Basic Bash Shell Commands," for more information on linked files). If the vim program is linked, it may be linked to a less than full-featured editor. Thus, it's a good idea to check for linked files.

On other distributions, you will find various flavors of the vim editor. Notice on this Ubuntu distribution that not only is there no alias for the `vi` command, but the `/usr/bin/vi` program file belongs to a series of file links:

```
$ alias vi
-bash: alias: vi: not found
$
$ which vi
/usr/bin/vi
$
$ ls -l /usr/bin/vi
lrwxrwxrwx 1 root root 20 Apr 23 14:33 /usr/bin/vi ->
 /etc/alternatives/vi
$
$ ls -l /etc/alternatives/vi
lrwxrwxrwx 1 root root 17 Apr 23 14:33 /etc/alternatives/vi ->
 /usr/bin/vim.tiny
$
$ readlink -f /usr/bin/vi
/usr/bin/vim.tiny
$
```

Thus, when the `vi` command is entered, the `/usr/bin/vim.tiny` program is executed. The `vim.tiny` program provides only a few vim editor features. If you are serious about trying out the vim editor and are using a distribution that uses a vim alternative, such as `vim.tiny` , consider installing the basic `vim` package for more vim features.

> **NOTE**
>
> **Notice in the preceding example that, instead of having to use the `ls -l` command multiple times to find a series of linked files' final object, you can use the `readlink -f` command. It immediately produces the linked file series' final object.**

Software package management was covered in detail in Chapter 9, “Installing Software.” Installing the basic `vim` package on this Ubuntu distribution is fairly straightforward:

```
$ sudo apt install vim
[sudo] password for christine:
[...]
The following additional packages will be installed:
  vim-runtime
Suggested packages:
  ctags vim-doc vim-scripts
The following NEW packages will be installed:
  vim vim-runtime
[...]
Do you want to continue? [Y/n] Y
[...]
Setting up vim (2:8.1.2269-1ubuntu5) ...
[...]
Processing triggers for man-db (2.9.1-1) ...
$
$ readlink -f /usr/bin/vi
/usr/bin/vim.basic
$
```

The basic vim editor is now installed on this Ubuntu distribution, and the `/usr/bin/vi` program file's link was automatically changed to point to `/usr/bin/vim.basic` . Thus, when the `vi` command is entered on this Ubuntu system, the basic vim editor is used instead of tiny vim.

Exploring vim basics

The vim editor works with data in a memory buffer. To start the vim editor, just type the `vim` command (or `vi` if there's an alias or linked file) and the name of the file you want to edit:

```
$ vi myprog.c
```

If you start vim without a filename, or if the file doesn't exist, vim opens a new buffer area for editing. If you specify an existing file on the command line, vim reads the entire file's contents into a buffer area, where it is ready for editing, as shown in Figure 10-1.

```
#include <stdio.h>

int main()
{
    int i;
    int factorial = 1;
    int number = 5;

    for(i = 1;  i <= number; i++)
    {
       factorial = factorial * i;
    }

    printf("The factorial of %d is %d\n", number, factorial);
    return 0;
}
~
~
~
~
~
~
~
~
~
~
~
"myprog.c" 16L, 248C                                    2,0-1         All
```

FIGURE 10-1 The vim main window

The vim editor detects the terminal type for the session (see Chapter 2, “Getting to the Shell”) and uses a full-screen mode to consume the entire console window for the editor area.

The initial vim edit window shows the contents of the file (if there are any) along with a message line at the bottom of the window. If the file contents don't take up the entire screen, vim places a tilde (~) on lines that are not part of the file (as shown in Figure 10-1).

The message line at the bottom indicates information about the edited file, depending on the file's status, and the default settings in your vim installation. If the file is new, the message `[New File]` appears.

The vim editor has three modes of operation:

- Command mode
- Ex mode
- Insert mode

When you first open a file (or start a new file) for editing, the vim editor enters *command* mode (sometimes called normal mode). In command mode, the vim editor interprets keystrokes as commands (more on those later).

In *insert* mode, vim places the letter, number, or symbol of the keys you type at the current cursor location in the buffer. To enter insert mode, enter `i` . To get out of insert mode and go back into command mode, press the Esc key on your keyboard.

In command mode, you can move the cursor around the text area by using the arrow keys (as long as your terminal type is detected properly by vim). If you happen to be on an unusual terminal connection that doesn't have the arrow keys defined, all hope is not lost. The vim editor include commands for moving the cursor:

- `h` to move left one character
- `j` to move down one line (the next line in the text)
- `k` to move up one line (the previous line in the text)
- `l` to move right one character

Moving around within large text files line by line can get tedious. Fortunately, vim provides a few commands to help speed things along:

- Press PageDown (or Ctrl+F) to move forward one screen of data.
- Press PageUp (or Ctrl+B) to move backward one screen of data.
- Enter `G` to move to the last line in the buffer.

- Enter *num* `G` to move to the line number *num* in the buffer.
- Enter *gg* to move to the first line in the buffer.

The vim editor has a special feature within command mode called *Ex mode*. This mode provides an interactive command line where you can enter additional commands to control the actions in vim. To get to Ex mode, press the colon key (`:`) in command mode. The cursor moves to the message line, and a colon (`:`) appears, waiting for you to enter a command.

Within the Ex mode are several commands for saving the buffer to the file and exiting vim:

- `q` to quit if no changes have been made to the buffer data
- `q!` to quit and discard any changes made to the buffer data
- `w` *filename* to save the file under a different filename
- `wq` to save the buffer data to the file and quit

After seeing just a few basic vim commands, you might understand why some people loathe the vim editor. To use vim to its fullest, you must know plenty of obscure commands. However, after you get a few of the basic vim commands down, you can quickly edit files directly from the command line, no matter what type of environment you're in. Due to its enormous functionality and despite its steep learning curve, the vim editor maintains its popularity. It is still considered one of the top 10 text editors.

Editing data

While in command mode, the vim editor provides several commands for editing the data in the buffer. Table 10-1 lists some common editing commands for vim.

TABLE 10-1 vim Editing Commands

Command	Description
`x`	Deletes the character at the current cursor position.
`dd`	Deletes the line at the current cursor position.
`dw`	Deletes the word at the current cursor position.
`d$`	Deletes to the end of the line from the current cursor position.
`J`	Deletes the line break at the end of the line at the current cursor position (joins lines).
`u`	Undoes the previous edit command.
`a`	Appends data after the current cursor position.
`A`	Appends data to the end of the line at the current cursor position.
`r` *char*	Replaces a single character at the current cursor position with *char*.
`R` *text*	Overwrites the data at the current cursor position with *text* , until you press Esc.

Some of the editing commands also allow you to use a numeric modifier to indicate how many times to perform the command. For example, the command `2x` deletes two characters, starting from the current cursor position, and the command `5dd` deletes five lines, starting at the line from the current cursor position.

NOTE

Be careful when trying to use the keyboard Backspace or Delete key while in the vim editor's command mode. The vim editor usually recognizes the Delete key only as the functionality of the `x` command, deleting the character at the current cursor location. Usually, the vim editor doesn't recognize the Backspace key in command mode as a deleting action, but instead as a method to move the cursor back one space.

Copying and pasting

A standard editor feature is the ability to cut or copy data and paste it elsewhere in the document. The vim editor provides a way to do this.

Cutting and pasting is relatively easy. You've already seen the commands in Table 10-1 that can remove data from the buffer. However, when vim removes data, it actually keeps it stored in a *separate* area. While in command mode, that data is retrievable by using the `p` command.

For example, use the `dd` command to delete a line of text, move the cursor to the buffer location where you want to place it, and then use the `p` command. The `p` command inserts the text after the line at the current cursor position. You can do this with any command that removes text while in command mode.

Copying text is a little bit trickier. The copy command in vim is `y` (for yank). You can use the same second character with `y` as with the `d` command (`yw` to yank a word, `y$` to yank to the end of a line). After you yank the text, move the

cursor to the location where you want to place the text and use the `p` command. The yanked text now appears at that location.

Yanking is tricky in that you can't see what happened because you're not affecting the text that you yank. You never know for sure what you yanked until you paste it somewhere. But there's another feature in vim that helps you out with yanking.

While in command mode, the *visual mode* highlights text as you move the cursor. You use visual mode to select text to yank for pasting. To enter visual mode, move the cursor to the location where you want to start yanking, and press `v` . Notice that the text at the cursor position is now highlighted. Next, move the cursor to cover the text you want to yank (you can even move down lines to yank more than one line of text). As you move the cursor, vim highlights the text in the yank area. After you've covered the text you want to copy, enter `y` to activate the yank command. Now that you have the text in the register, just move the cursor to where you want to paste and use the `p` command.

Searching and substituting

You can easily search for data in the buffer using the vim search command. To enter a search string, press the forward slash (/) key. The cursor goes to the message line, and vim displays a forward slash. Enter the text you want to find, and press the Enter key. The vim editor responds with one of three actions:

- If the word appears after the current cursor location, it jumps to the first location where the text appears.
- If the word doesn't appear after the current cursor location, it wraps around the end of the file to the first location in the file where the text appears (and indicates this with a message).
- It produces an error message stating that the text was not found in the file.

If the word appears, to continue searching for the same word, press the forward slash character and then press the Enter key, or you can use `n` , which stands for next.

> **NOTE**
>
> **Are you more familiar with writing scripts and programs on a Microsoft Windows platform with an integrated development environment (IDE)? If so, Microsoft's Visual Studio Code is available on Linux. (We'll wait a minute for you to reread that last sentence.) Yes, Microsoft offers Visual Studio for Linux. Find information for installing it on your particular Linux distribution at `code.visualstudio.com/docs/setup/linux`. If desired, you can add the VSCodeVim plugin to Visual Studio, and have all the vim commands at your fingertips.**

The substitute command, performed within Ex mode, allows you to quickly replace (substitute) one word for another in the text. To get to the substitute command, you must be in command-line mode. The format for the substitute command is `:s/` *`old`* `/` *`new`* `/` . The vim editor jumps to the first occurrence of the text *`old`* and replaces it with the text *`new`* . You can make a few modifications to the substitute command to replace more than one occurrence of the text:

- `:s/` *`old`* `/` *`new`* `/g` to replace all occurrences of *`old`* in a line
- `:` *`n`* `,` *`ms`* `/` *`old`* `/` *`new`* `/g` to replace all occurrences of *`old`* between line numbers *`n`* and *`m`*
- `:%s/` *`old`* `/` *`new`* `/g` to replace all occurrences of *`old`* in the entire file
- `:%s/` *`old`* `/` *`new`* `/gc` to replace all occurrences of *`old`* in the entire file, but prompt for each occurrence

As you can see, for a console mode text editor, vim contains quite a few advanced features. Because nearly every Linux distribution includes it, it's a good idea to at least know the basics of the vim editor so that you can always edit scripts, no matter where you are or what you have available.

Navigating the nano Editor

In contrast to vim, which is a complicated editor with powerful features, nano is a simple editor. For individuals who need a simple console mode text editor that is easy to navigate, nano is the tool to use. It's also a great text editor for those who are just starting on their Linux command-line adventure.

The nano text editor is a clone of the Unix systems' Pico editor. Although Pico also is a light and simple text editor, it is not licensed under the GPL. Not only is the nano text editor licensed under the GPL, it is also part of the GNU project.

The nano text editor is installed on most Linux distributions by default. Everything about the nano text editor is easy. To open a file at the command line with nano, enter

```
$ nano myprog.c
```

If you start nano without a filename, or if the file doesn't exist, nano simply opens a new buffer area for editing. If you specify an existing file on the command line, nano reads the entire contents of the file into a buffer area, where it is ready for editing, as shown in Figure 10-2.

```
  GNU nano 4.8                    myprog.c
#include <stdio.h>

int main()
{
    int i;
    int factorial = 1;
    int number = 5;

    for(i = 1;  i <= number; i++)
    {
       factorial = factorial * i;
    }

    printf("The factorial of %d is %d\n", number, factorial);
    return 0;
}

                              [ Read 16 lines ]
^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^J Justify
^X Exit        ^R Read File   ^\ Replace     ^U Paste Text  ^T To Spell
```

FIGURE 10-2 The nano editor window

Notice that at the bottom of the nano editor window, various commands with brief descriptions are shown. These commands are the nano control commands. The caret (^) symbol shown represents the Ctrl key. Therefore, ^X stands for the keyboard sequence Ctrl+X.

> **TIP**
>
> **Though the nano control commands list capital letters in the keyboard sequences, you can use either lowercase or uppercase characters for control commands.**

Having most of the basic commands listed right in front of you is great — no need to memorize what control command does what. Table 10-2 presents the various nano control commands.

TABLE 10-2 nano Control Commands

Command	Description
Ctrl+C	Displays the cursor's position within the text editing buffer.
Ctrl+G	Displays nano's main help window.
Ctrl+J	Justifies the current text paragraph.
Ctrl+K	Cuts the text line and stores it in cut buffer.
Ctrl+O	Writes out the current text editing buffer to a file.
Ctrl+R	Reads a file into the current text editing buffer.
Ctrl+T	Starts the available spell checker.
Ctrl+U	Pastes text stored in cut buffer and places in current line.
Ctrl+V	Scrolls text editing buffer to next page.
Ctrl+W	Searches for word or phrases within text editing buffer.
Ctrl+X	Closes the current text editing buffer, exits nano, and returns to the shell.
Ctrl+Y	Scrolls text editing buffer to previous page.

The control commands listed in Table 10-2 are really all you need. However, if you desire more powerful control

features than those listed, nano has them. To see more control commands, press Ctrl+G in the nano text editor to display its main help window containing additional control commands.

> **NOTE**
>
> **Some of these additional commands available in nano are called Meta-key sequences. In the nano documentation, they are denoted by the letter M. For example, you'll find the key sequence to undo the last task denoted as `M-U` in the nano help system. But don't press the M key to accomplish this. Instead, M represents either the Esc, Alt, or Meta key, depending on your keyboard's configuration. Thus, you might press the Alt+U key combination to undo the last task within nano.**

Even more features are available through command-line options to control the nano editor. Creating a backup file before editing is one nice selection. Type **`man nano`** to see these additional command-line options for starting nano.

The vim and nano text editors offer a choice between powerful and simple console mode text editors. However, neither offers the ability to use graphical features for editing. Some text editors can operate in both worlds, as explored in the next section.

Exploring the Emacs Editor

The Emacs editor was an extremely popular editor for Digital Equipment Corporation (DEC) computers in the late 1970s. Developers liked it so much that they ported it to the Unix environment, and then to the Linux environment, where its official name is GNU Emacs. Though currently not as popular as vim, it still has its place in the world.

The Emacs editor started out life as a console editor, much like vim, but was migrated to the graphical world. The original console mode editor is still available, but it can use a graphical window to allow editing text in a graphical environment. Typically, when you start the Emacs editor from a command line, if the editor determines you have an available graphical session, it starts in graphical mode. If you don't, it starts in console mode.

This section describes both the console mode and graphical mode Emacs editors so that you'll know how to use either one if you want (or need) to.

Checking your Emacs package

Many distributions do not come with the Emacs editor installed by default. You can check your Red Hat–based distribution by using the `which` and/or `dnf list` (use `yum list` on older versions of Red Hat-based distributions) command, as shown on this CentOS distribution:

```
$ which emacs
/usr/bin/which: no emacs in (/home/christine/.local/bin:
/home/christine/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:
/usr/sbin)
$
$ dnf list emacs
[...]
Available Packages
emacs.x86_64[...]
$
```

The `emacs` editor package is not currently installed on this CentOS distribution. However, it is available to be installed. (For a more thorough discussion on displaying installed software, see Chapter 9.)

For a Debian-based distribution, check for the Emacs editor package by using the `which` and/or `apt show` command, as shown on this Ubuntu distribution:

```
$ which emacs
$
$ apt show emacs
Package: emacs
[...]
Description: GNU Emacs editor (metapackage)
 GNU Emacs is the extensible self-documenting text editor.
 This is a metapackage that will always depend on the latest
 recommended Emacs variant (currently emacs-gtk).

$
```

The `which` command operates a little differently here. When it does not find the installed command, it simply returns the Bash shell prompt. The `emacs` editor package for this Ubuntu distribution is available to be installed. The following shows the Emacs editor being installed on Ubuntu:

```
$ sudo apt install emacs
[sudo] password for christine:
Reading package lists... Done
[...]
Do you want to continue? [Y/n] Y
```

```
[...]
$
$ which emacs
/usr/bin/emacs
$
```

Now when the `which` command is used, it points to the `emacs` program file. The Emacs editor is ready for use on this Ubuntu distribution.

For the CentOS distribution, install the Emacs editor using the `dnf install` or `yum install` command:

```
$ sudo yum install emacs
[sudo] password for christine:
[...]
Dependencies resolved.
[...]
Is this ok [y/N]: Y
Downloading Packages:
[...]
Complete!
$
$ which emacs
/usr/bin/emacs
$
```

With the Emacs editor successfully installed on your Linux distribution, you can begin to explore its different features, starting with using it on the console.

Using Emacs on the console

The console mode version of Emacs is another editor that uses lots of key commands to perform editing functions. The Emacs editor uses key combinations involving the Ctrl key and the Meta key. In most terminal emulator packages, the Meta key is mapped to the Alt key.

The official Emacs documents abbreviate the Ctrl key as C- and the Meta key as M-. Thus, the Ctrl+X key combination is shown in the document as C-x. This chapter section uses the Emacs' documentation format.

Exploring the basics of Emacs

To edit a file using Emacs, from the command line, enter

```
$ emacs myprog.c
```

The Emacs console mode window appears and loads the file into the active buffer, as shown in Figure 10-3.

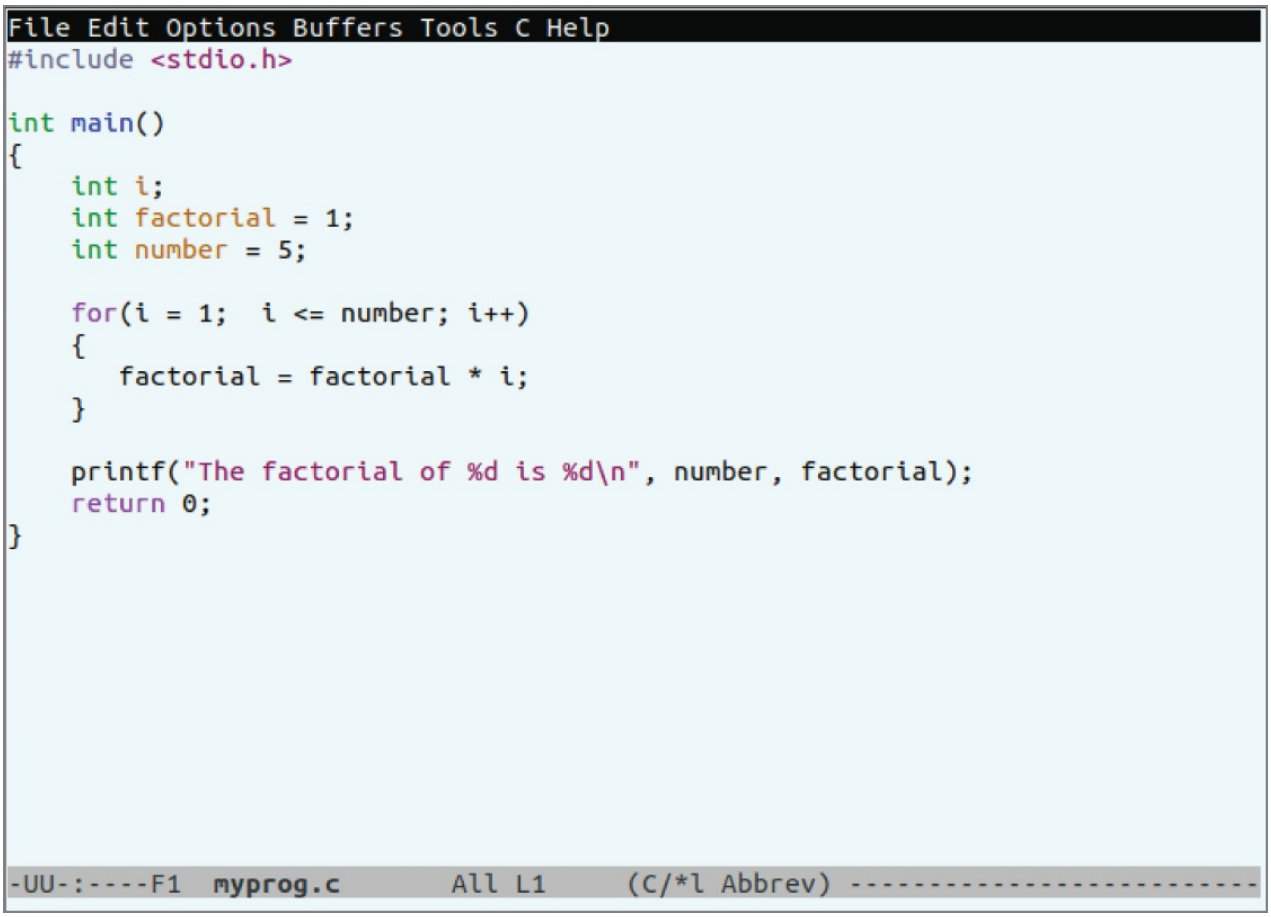

FIGURE 10-3 Editing a file using the Emacs editor in console mode

You'll notice that the top of the console mode window shows a typical menu bar. Unfortunately, you can't use the menu bar in console mode, only in graphical mode.

> **NOTE**
>
> **If you run Emacs in a graphical desktop environment, some commands in this section work differently than described. To use Emac's console mode in a graphical desktop environment, use the `emacs -nw` command. If you want to use Emacs's graphical features, see the section "Using Emacs in a GUI."**

Unlike the vim editor, where you move into and out of insert mode and switch between entering commands and inserting text, the Emacs editor has only one mode. If you type a printable character, Emacs inserts it at the current cursor position. If you type a command, Emacs executes the command.

To move the cursor around the buffer area, you can use the arrow keys and the PageUp and PageDown keys, assuming that Emacs detected your terminal emulator correctly. If not, these commands move the cursor around:

- `C-p` moves up one line (the previous line in the text).
- `C-b` moves left (back) one character.
- `C-f` moves right (forward) one character.
- `C-n` moves down one line (the next line in the text).

The following commands make longer jumps within the text:

- `M-f` moves right (forward) to the next word.
- `M-b` moves left (backward) to the previous word.
- `C-a` moves to the beginning of the current line.
- `C-e` moves to the end of the current line.
- `M-a` moves to the beginning of the current sentence.
- `M-e` moves to the end of the current sentence.
- `M-v` moves back one screen of data.
- `C-v` moves forward one screen of data.
- `M-<` moves to the first line of the text.
- `M->` moves to the last line of the text.

You should know these commands for saving the editor buffer back into the file and exiting Emacs:

- `C-x C-s` saves the current buffer contents to the file.
- `C-z` exits Emacs but keeps it running in your session so you can come back to it.
- `C-x C-c` exits Emacs and stops the program.

You'll notice that two of these features require two key combinations. The `C-x` command is called the *extend command*. This provides yet another whole set of commands to work with.

Editing data

The Emacs editor is pretty robust about inserting and deleting text in the buffer. To insert text, just move the cursor to the location where you want to insert the text and start typing.

To delete text, Emacs uses the Backspace key to delete the character before the current cursor position and the Delete key to delete the character at the current cursor location.

The Emacs editor also has commands for cutting text. The Emacs documentation calls this *killing text*, but we'll stick with the friendlier *cutting* terminology.

The difference between deleting text and cutting text is that when you cut text, Emacs places it in a temporary area where you can retrieve it (see the next section, "Copying and Pasting"). Deleted text is gone forever.

These commands are for cutting text in the buffer:

- `M-Backspace` cuts the word before the current cursor position.
- `M-d` cuts the word after the current cursor position.
- `C-k` cuts from the current cursor position to the end of the line.
- `M-k` cuts from the current cursor position to the end of the sentence.

> **TIP**
>
> **If you happen to make a mistake when cutting text, the `C-/` command undoes the cut command and returns the data to the state it was in before you cut it.**

The Emacs editor also includes a fancy way of mass-cutting text. Just move the cursor to the start of the area you want to cut, and press either the C-@ or C-spacebar keys. Then move the cursor to the end of the area you want to cut, and enter `C-w` . All the text between the two locations is cut.

Copying and pasting

You've seen how to cut data from the Emacs buffer area; now it's time to see how to paste it somewhere else. Unfortunately, if you use the vim editor, this process may confuse you when you use the Emacs editor.

In an unfortunate coincidence, pasting data in Emacs is called *yanking*. In the vim editor, copying is called yanking, which is what makes this a difficult thing to remember if you happen to use both editors.

After you cut data using one of the cut commands, move the cursor to the location where you want to paste the data, and use the `C-y` command. This yanks the text out of the temporary area and pastes it at the current cursor position. The `C-y` command yanks the text from the last cut command. If you've performed multiple cut commands, you can cycle through them using the `M-y` command.

To copy text, just yank it back into the same location you cut it from and then move to the new location and use the `C-y` command again. You can yank text back as many times as you desire.

Searching and replacing

Searching for text in the Emacs editor is done by using the `C-s` and `C-r` commands. The `C-s` command performs a forward search in the buffer area from the current cursor position to the end of the buffer, whereas the `C-r` command performs a backward search in the buffer area from the current cursor position to the start of the buffer.

When you enter either the `C-s` or the `C-r` command, a prompt appears in the bottom line, querying you for the text to search. You can perform two types of searches in Emacs.

In an *incremental* search, the Emacs editor performs the text search in real-time mode as you type the word. When you type the first letter, it highlights all the occurrences of that letter in the buffer. When you type the second letter, it highlights all the occurrences of the two-letter combination in the text and so on until you complete the text you're searching for.

In a *non-incremental* search, press the Enter key after the `C-s` or `C-r` command. This locks the search query into the bottom line area and allows you to type the search text in full before searching.

To replace an existing text string with a new text string, you must use the `M-x` command. This command requires a text command, along with parameters.

The text command is `replace-string` . After typing the command, press the Enter key, and Emacs queries you for the existing text string. After entering that, press the Enter key again and Emacs queries you for the new replacement text string.

Using buffers in Emacs

The Emacs editor allows you to edit multiple files at the same time by having multiple buffer areas. You can load files into a buffer and switch between buffers while editing.

To load a new file into a buffer while you're in Emacs, use the `C-x C-f` commands. This is the Emacs find-file mode, called *Dired*. It takes you to the bottom line in the window and allows you to enter the name of the file you want to start to edit. If you don't know the name or location of the file, just press the Enter key. This brings up a file browser in the edit window, as shown in Figure 10-4.

```
File Edit Options Buffers Tools Operate Mark Regexp Immediate Subdir Help
  /home/christine:
  total used in directory 120 available 11645800
  drwxr-xr-x 17 christine christine  4096 May  8 16:05 .
  drwxr-xr-x  4 root      root       4096 Apr 23 15:12 ..
  -rw-------  1 christine christine   828 May  8 14:11 .bash_history
  -rw-r--r--  1 christine christine   220 Feb 25 07:03 .bash_logout
  -rw-r--r--  1 christine christine  3782 May  7 13:55 .bashrc
  drwx------ 14 christine christine  4096 May  1 15:14 .cache
  drwxr-xr-x 13 christine christine  4096 May  1 15:06 .config
  drwxr-xr-x  2 christine christine  4096 Apr 23 15:13 Desktop
  drwxr-xr-x  2 christine christine  4096 Apr 23 15:13 Documents
  drwxr-xr-x  2 christine christine  4096 Apr 23 15:13 Downloads
  drwx------  3 christine christine  4096 May  8 15:35 .emacs.d
  drwx------  3 christine christine  4096 May  2 16:35 .gnupg
  drwxr-xr-x  3 christine christine  4096 Apr 23 15:13 .local
  drwx------  5 christine christine  4096 May  1 15:14 .mozilla
  drwxr-xr-x  2 christine christine  4096 Apr 23 15:13 Music
  -rwxrwxr-x  1 christine christine 16688 May  7 14:26 myprog
  -rw-rw-r--  1 christine christine   248 May  8 16:05 #myprog.c#
  lrwxrwxrwx  1 christine christine    34 May  8 16:03 .#myprog.c -> christine\
@UDesktop.3662:1588962380
  -rw-rw-r--  1 christine christine   248 May  8 11:11 myprog.c
  -rw-------  1 christine christine    20 May  7 14:29 myprog.c.save
  drwxr-xr-x  2 christine christine  4096 Apr 23 15:13 Pictures
  -rw-r--r--  1 christine christine   807 Feb 25 07:03 .profile
  drwxr-xr-x  2 christine christine  4096 Apr 23 15:13 Public
-UUU:%%--F1  christine          Top L3     (Dired by name) -------------------
```

FIGURE 10-4 The Emacs file browser

From here, you can browse to the file you want to edit. To traverse up a directory level, go to the double dot entry and press the Enter key. To traverse down a directory, go to the directory entry and press Enter. When you've found the file you want to edit, press Enter and Emacs loads it into a new buffer area.

> **TIP**
>
> **When you start the file browser in the edit window, you may decide you don't want to open a file. In this case, enter `q` to quit the file browser window.**

You can list the active buffer areas by entering the `C-x C-b` extended command combination. The Emacs editor splits the editor window and displays a list of buffers in the bottom window. Emacs provides two buffers in addition to your main editing buffer:

- A scratch area called scratch
- A message area called Messages

The scratch area allows you to enter LISP programming commands as well as enter notes to yourself. The messages area shows messages generated by Emacs while operating. If any errors occur while using Emacs, they appear in the messages area.

You can switch to a different buffer area in the window in two ways:

- Use `C-x C-b` to open the buffer listing window. Use `C-x b` and then type **`*Buffer List*`** to switch to that window. Use the arrow keys to move the cursor to the buffer area you want and press the Enter key.
- Use `C-x b` to type in the name of the buffer area you want to switch to.

When you select the option to switch to the buffer listing window, Emacs opens the buffer area in a new window area. The Emacs editor allows you to have multiple windows open in a single session. The following section discusses how to manage multiple windows in Emacs.

Using windows in console mode Emacs

The console mode Emacs editor was developed many years before the idea of graphical windows appeared. However, it was advanced for its time in that it could support multiple editing windows within the main Emacs window.

You can split the Emacs editing window into multiple windows by using one of two commands:

- `C-x 2` splits the window horizontally into two windows.
- `C-x 3` splits the window vertically into two windows.

To move from one window to another, use the `C-x o` command. Notice that when you create a new window, Emacs uses the buffer area from the original window in the new window. After you move into the new window, you can use the `C-x C-f` command to load a new file or use one of the commands to switch to a different buffer area in the new window.

To close a window, move to it and use the `C-x 0` (that's a zero) command. If you want to close all the windows except the one you're in, use the `C-x 1` (that's a numerical one) command.

Using Emacs in a GUI

If you use Emacs from a GUI environment (such as in the GNOME Shell desktop), it starts in graphical mode, as shown in Figure 10-5.

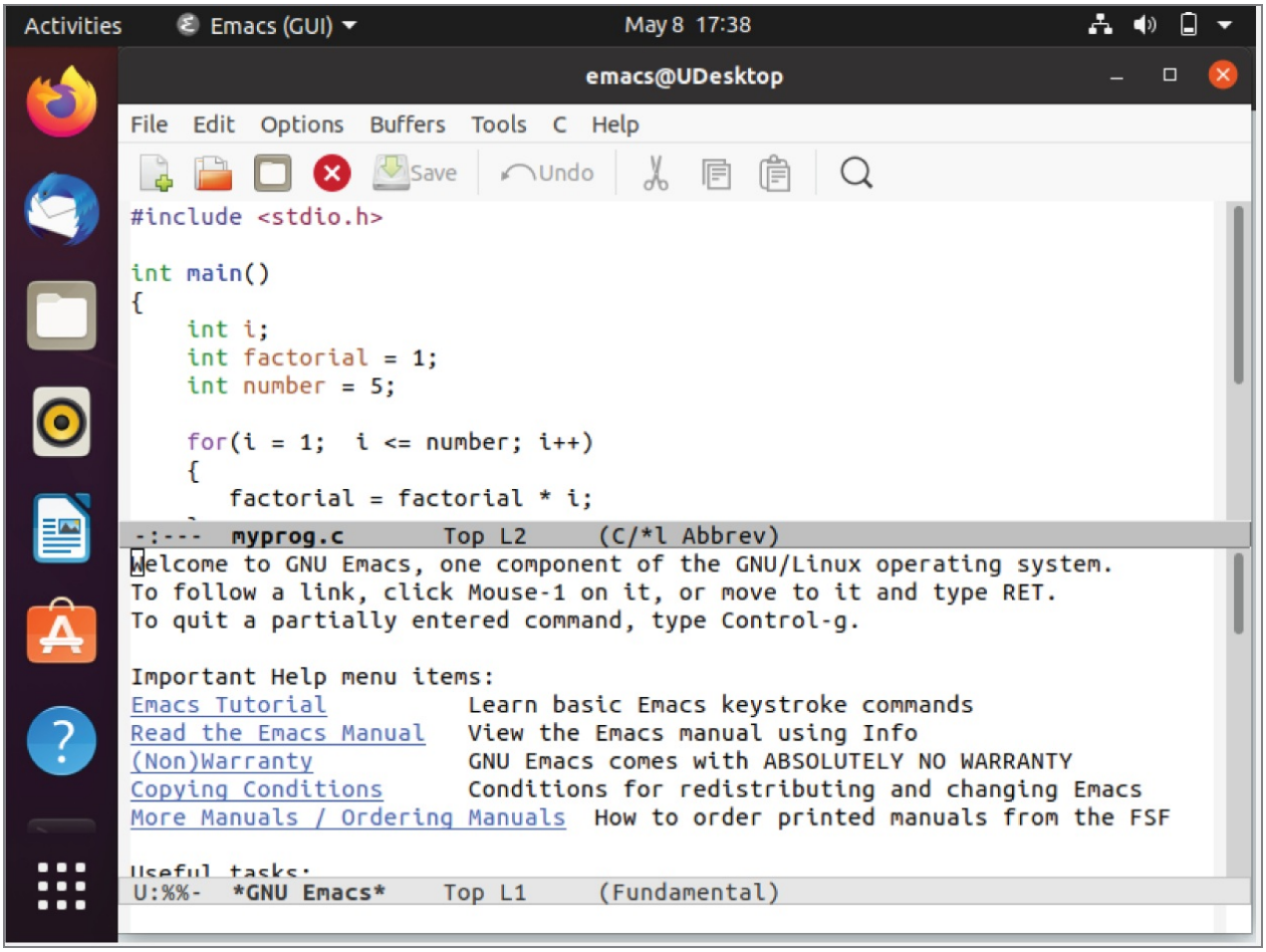

FIGURE 10-5 The Emacs graphical window

If you've already used Emacs in console mode, you should be fairly familiar with the graphical mode. All the key commands are available as menu bar items. The Emacs menu bar contains the following items:

- **File** allows you to open files in the window, create new windows, close windows, save buffers, and print buffers.
- **Edit** allows you to cut and copy selected text to the clipboard, paste clipboard data to the current cursor position, search for text, and replace text.
- **Options** provides settings for many more Emacs features, such as highlighting, word wrap, cursor type, and setting fonts.
- **Buffers** lists the current buffers available and allows you to easily switch between buffer areas.
- **Tools** provides access to the advanced features in Emacs, such as the command-line interface access, spell checking, comparing text between files (called diff), sending an email message, calendar, and the calculator.
- **C** allows advanced settings for highlighting C program syntax, compiling, running, and debugging the code.
- **Help** provides the Emacs manual online for access to help on specific Emacs functions.

The graphical Emacs window is an example of an older console application that made the migration to the graphical world. Now that many Linux distributions provide graphical desktops (even on servers that don't need them), graphical editors are becoming more commonplace. Popular Linux desktop environments (such as KDE Plasma and GNOME Shell) have also provided graphical text editors specifically for their environments, which are covered in the

rest of this chapter.

Exploring the KDE Family of Editors

If you're using a Linux distribution that uses the KDE Plasma desktop environment, you have a couple of options when it comes to text editors. The KDE project officially supports two popular text editors:

- **KWrite:** A single-screen text-editing package
- **Kate:** A full-featured, multiwindow text-editing package

Both of these editors are graphical text editors that contain many advanced features. The Kate editor also provides extra niceties not often found in standard text editors. This section describes each of the editors and shows some of the features you can use to help with your shell script editing.

Looking at the KWrite editor

The basic editor for the KDE Plasma environment is KWrite. It provides simple word processing–style text editing, along with support for code syntax highlighting and editing. The default KWrite editing window is shown in Figure 10-6.

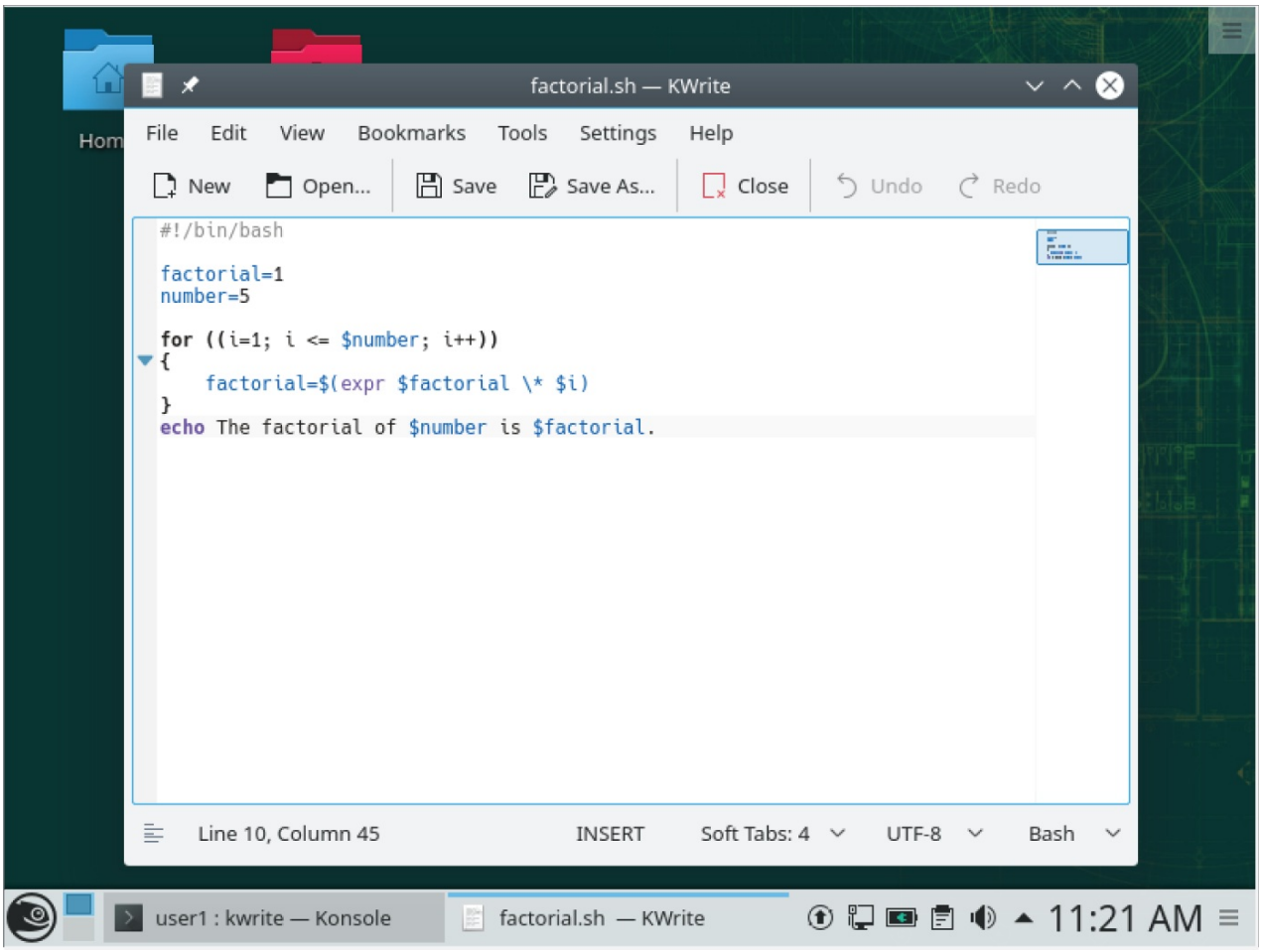

FIGURE 10-6 The default KWrite window editing a shell script program

You can't tell from Figure 10-6, but the KWrite editor recognizes several types of programming languages and uses color coding to distinguish constants, functions, and comments. The KWrite editing window provides full cut and paste capabilities, using the mouse and the arrow keys. As with a word processor, you can highlight and cut (or copy) text anywhere in the buffer area and paste it at any other place.

> **TIP**
>
> **Typically, KWrite is no longer installed by default on the KDE desktop environment. However, you can easily install it (see Chapter 9) on Plasma or other desktop environments, where available. The package name is `kwrite`.**

To edit a file using KWrite, you can either select KWrite from the KDE menu system on your desktop (some Linux distributions even create a Panel icon for it) or start it from the command-line prompt:

```
$ kwrite factorial.sh
```

The `kwrite` command has several command-line parameters you can use to customize how it starts. Here are few of the more practical ones:

- `--stdin` causes KWrite to read data from the standard input device instead of a file.
- `--encoding` specifies a character encoding type to use for the file.
- `--line` specifies a line number in the file to start at in the editor window.
- `--column` specifies a column number in the file to start at in the editor window.

The KWrite editor provides both a menu bar and a toolbar at the top of the edit window, allowing you to select features and change the configuration settings of the KWrite editor.

The menu bar contains these items:

- **File** loads, saves, prints, and exports text from files.
- **Edit** manipulates text in the buffer area.
- **View** manages how the text appears in the editor window.
- **Bookmarks** handle pointers to return to specific locations in the text; this option may need to be enabled in the configurations.
- **Tools** contains specialized features to manipulate the text.
- **Settings** configures the way the editor handles text.
- **Help** gives you information about the editor and commands.

The Edit menu bar item provides commands for all your text-editing needs. Instead of having to remember cryptic key commands (which by the way, KWrite also supports), you can just select items in the Edit menu bar, as shown in Table 10-3.

TABLE 10-3 The KWrite Edit Menu Items

Item	Description
Undo	Reverses the last action or operation.
Redo	Reverses the last undo action.
Cut	Deletes the selected text and places it in the clipboard.
Copy	Copies the selected text to the clipboard.
Paste	Inserts the current contents of the clipboard at the current cursor position.
Clipboard History	Displays portions of text recently copied to the clipboard from which you can select to paste.
Copy As HTML	Copies the selected text as HTML.
Select All	Selects all text in the editor.
Deselect	Deselects any text that is currently selected.
Block Selection Mode	Toggles on/off block selection mode which allows vertical text selection.
Input Modes	Toggles between a normal and a vi-like editing mode.
Overwrite Mode	Toggles insert mode to overwrite mode, replacing text with new typed text instead of just inserting the new text.
Find	Produces the Find Text dialog box, which allows you to customize a text search.
Find Variants	Provides a submenu of various text searches — Find Next, Find Previous, Find Selected, and Find Selected Backwards.
Replace	Produces the Replace With dialog box, which allows you to customize a text search and replace.
Go To	Provides a submenu of various Go To choices — Move To Matching Bracket, Select To Matching Bracket, Move To Previous Modified Line, Move To Next Modified Line, Go To Line.

The Find feature has two modes. Normal mode performs simple text searches and power searches. Replace mode lets you do advanced searching and replacing if necessary. You toggle between the two modes using the icon on the lower-right side of the window, as shown in Figure 10-7.

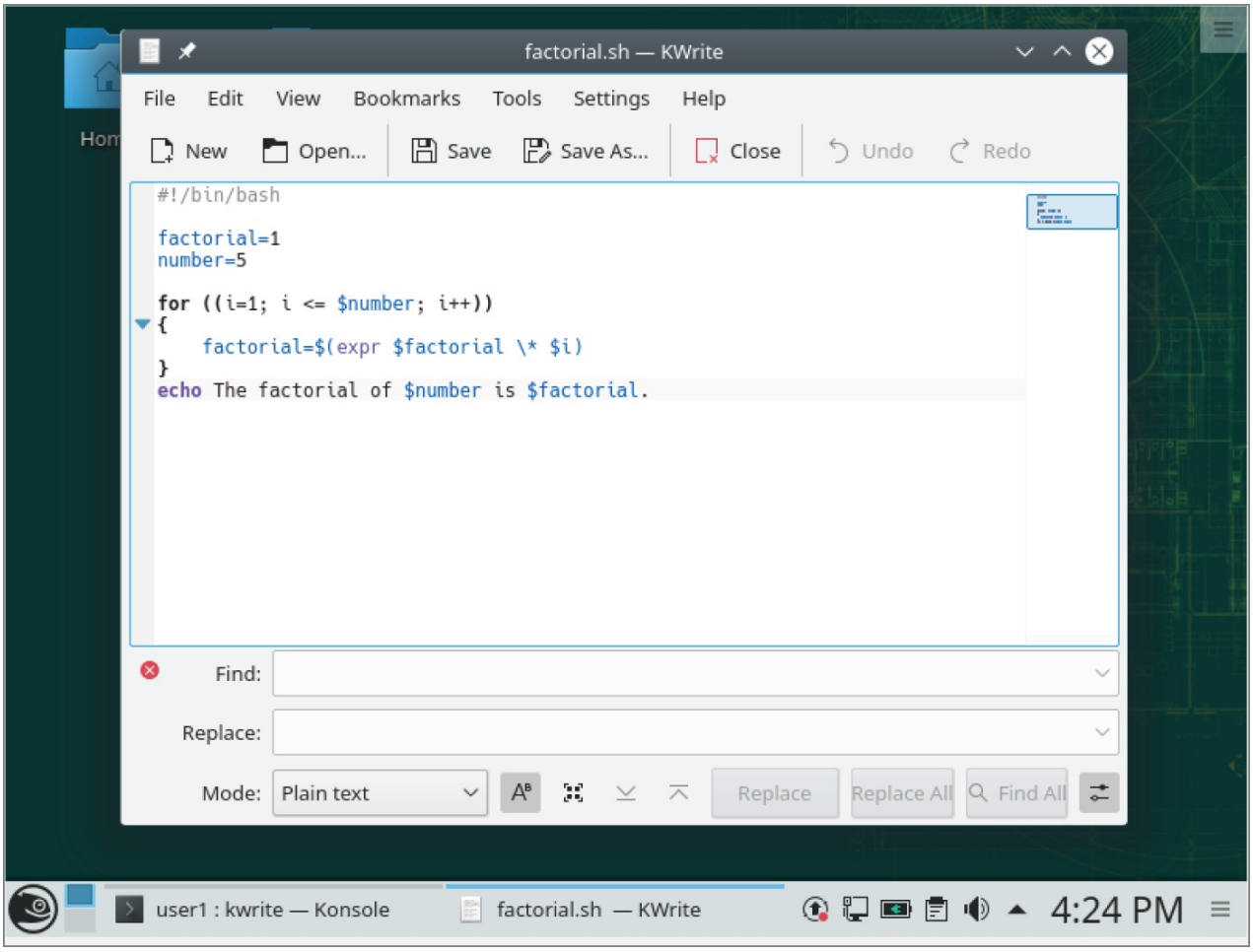

FIGURE 10-7 The KWrite Find section

The Find power mode allows you to search not only with words, but with a regular expression (discussed in Chapter 20, "Regular Expressions") for the search. You can use some other options to customize the search as well, indicating, for example, whether or not to perform a case-sensitive search or to look only for whole words instead of finding the text within words.

The Tools menu bar item provides several handy features for working with the text in the buffer area. Table 10-4 describes the tools available in KWrite.

TABLE 10-4 The KWrite Tools

Tool	Description
Read Only Mode	Locks the text so that no changes can be made while in the editor.
Mode	Sets the file type arrangement for the text from a submenu selection.
Highlighting	Selects the text highlighting plan from a submenu selection.
Indentation	Sets the indentation style for the text from a submenu selection.
Encoding	Chooses the character set encoding used by the text.
End of Line	Switches the End of Line characters between Unix, Windows/DOS, and Macintosh.
Add Byte Order Mark	Toggles on/off setting a byte order mark (BOM) at the start of the text.
Scripts	Selects scripted actions from a submenu for quickly accomplishing such items as editing.
Invoke Code Completion	Displays a tooltip suggesting the code text to use at the cursor's location; autocompletion using the tip is selected by pressing Enter.
Word Completion	Performs autocompletion of the current typed text from a submenu selection.
Spelling	Starts and/or controls the spell-check program for the text.
Clean Indentation	Returns all paragraph indentation to the original settings.
Align	Forces the current line or the selected lines to return to the default indentation settings.
Toggle Comment	Turns the text line into a comment line using syntax based on the current mode selected.
Uppercase	Sets the selected text, or the character at the current cursor position, to uppercase.
Lowercase	Sets the selected text, or the character at the current cursor position, to lowercase.
Capitalize	Capitalizes the first letter of the selected text or the word at the current cursor position.
Join Lines	Combines the selected lines, or the line at the current cursor position and the next line, into one line.
Apply Word Wrap	Enables word wrapping in the text. If a line extends past the editor window edge, the line continues on the next line.

There are lots of tools for this simple text editor! The Mode and Indentation tools are particularly nice to help you along if you are writing a script or program. The Mode's Script submenu is displayed in Figure 10-8.

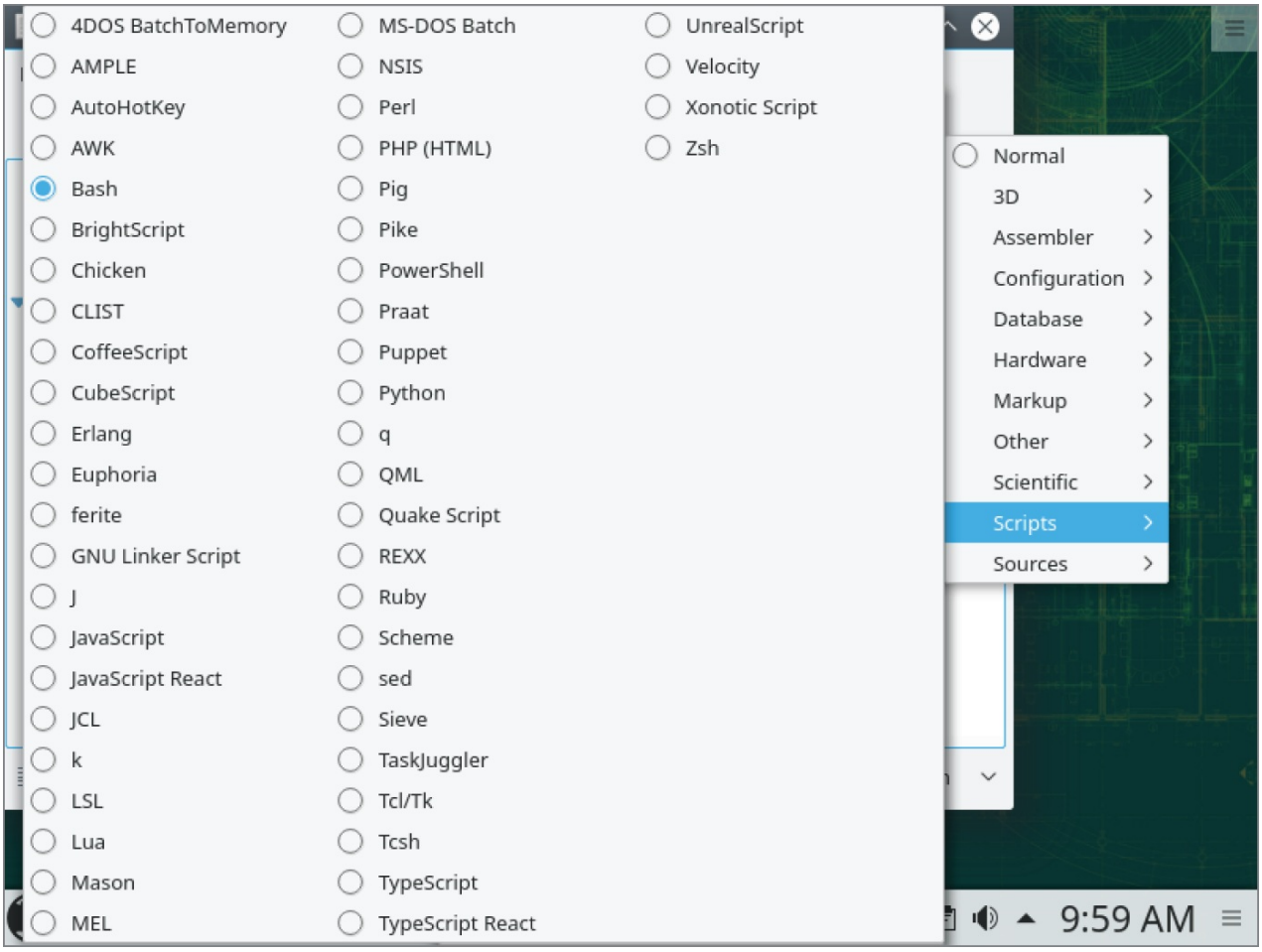

FIGURE 10-8 The KWrite Tool Mode Script submenu

The Settings menu includes the Configure Editor dialog box, shown in Figure 10-9.

The Configuration dialog box uses icons on the left side for you to select the feature in KWrite to configure. When you select an icon, the right side of the dialog box shows the configuration settings for the feature.

The Appearance feature allows you to set several features that control how the text appears in the text editor window. You can enable word wrap, line count (great for programmers), and word count from here. With the Fonts & Colors feature, you can customize the complete color scheme for the editor, determining what colors to make each category of text in the program code. There are also several customizations you can choose, such as encoding and mode, so you don't have to set them via the menu system each time you open a file.

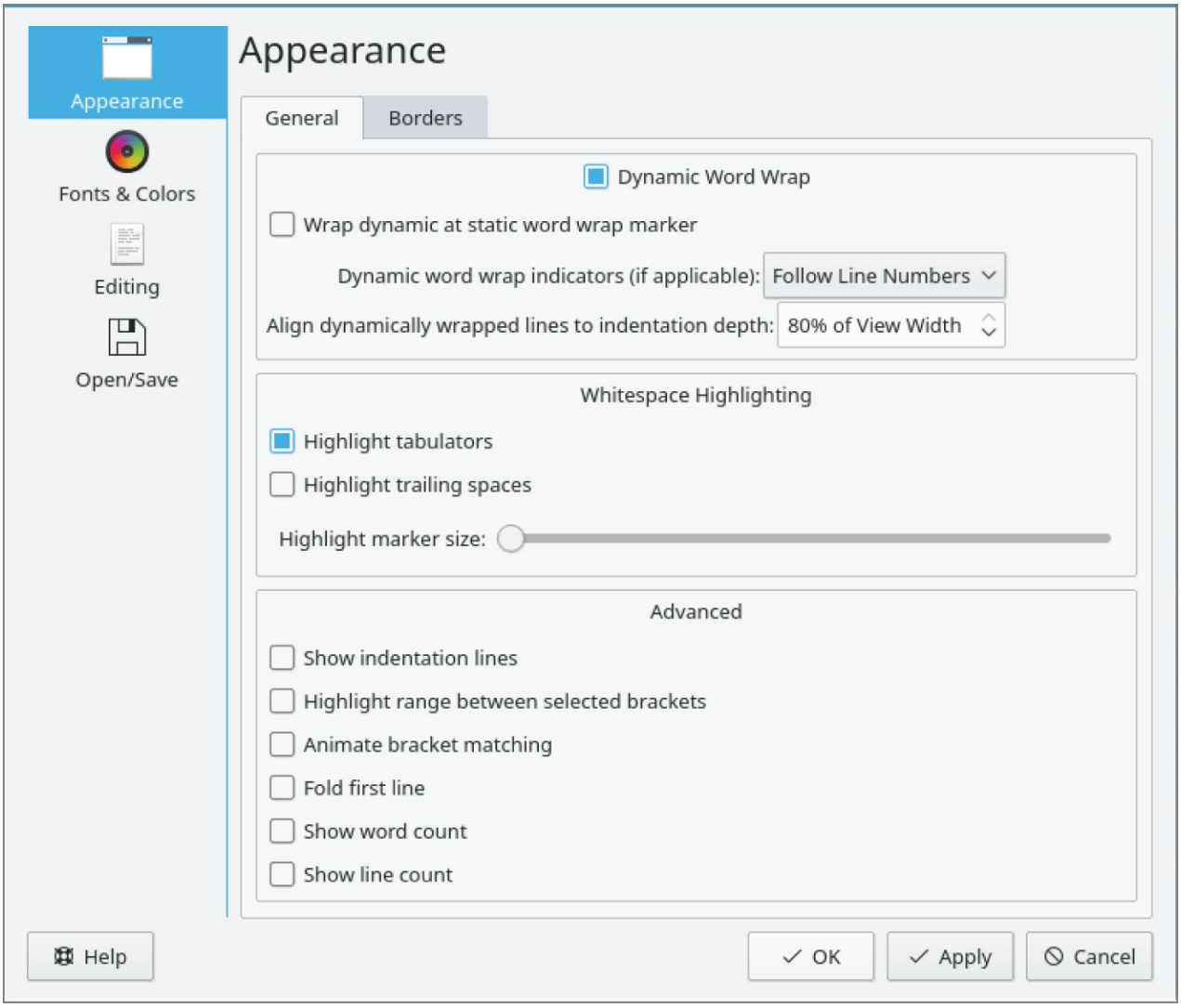

FIGURE 10-9 The KWrite Configure Editor dialog box

Looking at the Kate editor

The Kate editor is the flagship editor for the KDE Project. It uses the same core text editor as the KWrite application (so most of those features are the same), but it incorporates lots of other features into a single package, including a multiple document interface (MDI).

> **TIP**
>
> **If you find that the Kate editor has not been installed with your KDE desktop environment, you can easily install it (see Chapter 9). The package name that contains Kate is `kate` or `kdesdk` .**

When you start the Kate editor from the Plasma menu system, you see the main Kate editor window, shown in Figure 10-10.

You'll notice the window looks very similar to the KWrite editing window shown previously in Figure 10-6. However, there are differences. For example, the left side frame shows the Documents icon. Clicking this icon opens a new interface called the Documents List, shown in Figure 10-11, which allows switching between open documents, creating new documents, and exploring other files to open.

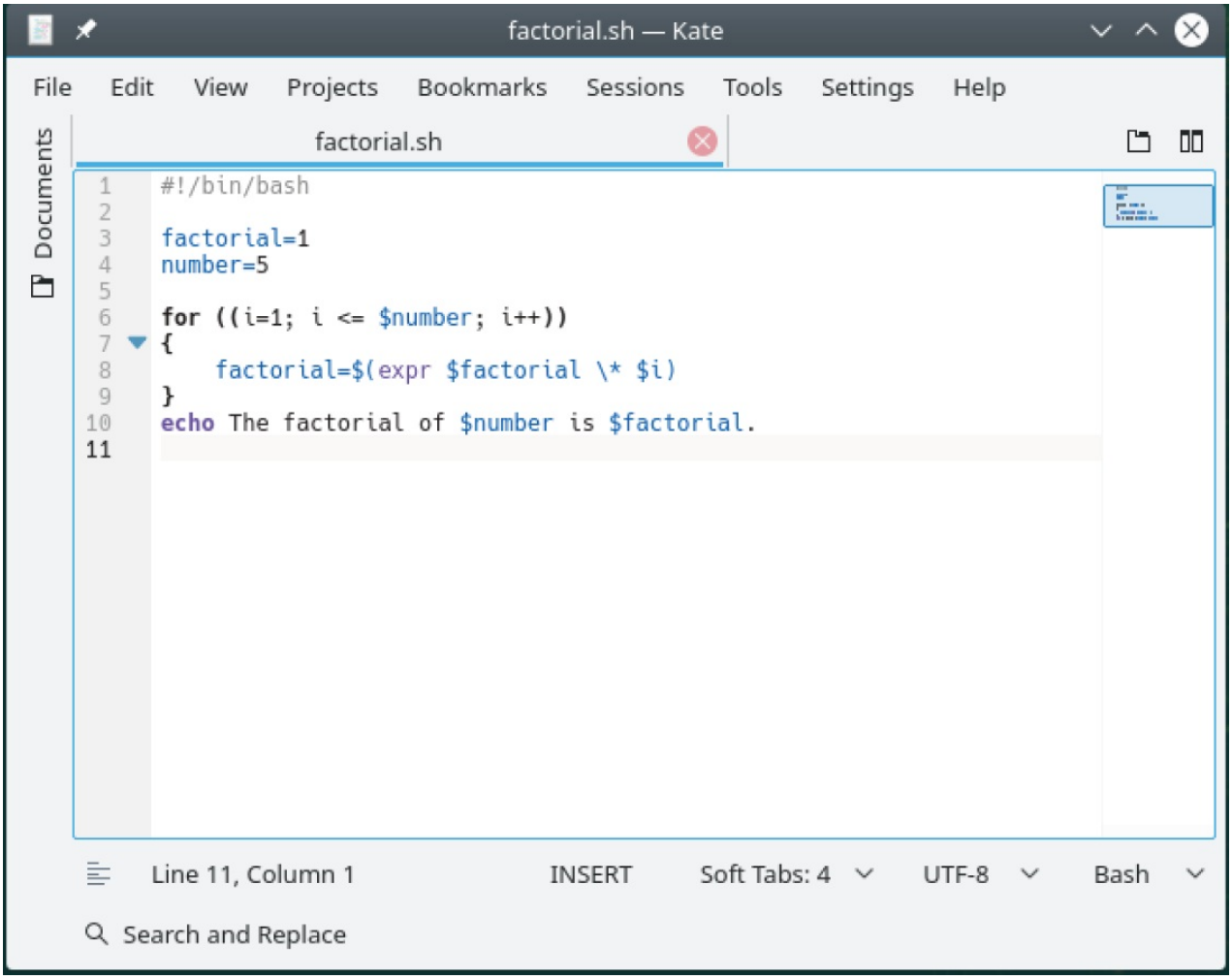

FIGURE 10-10 The main Kate editing window

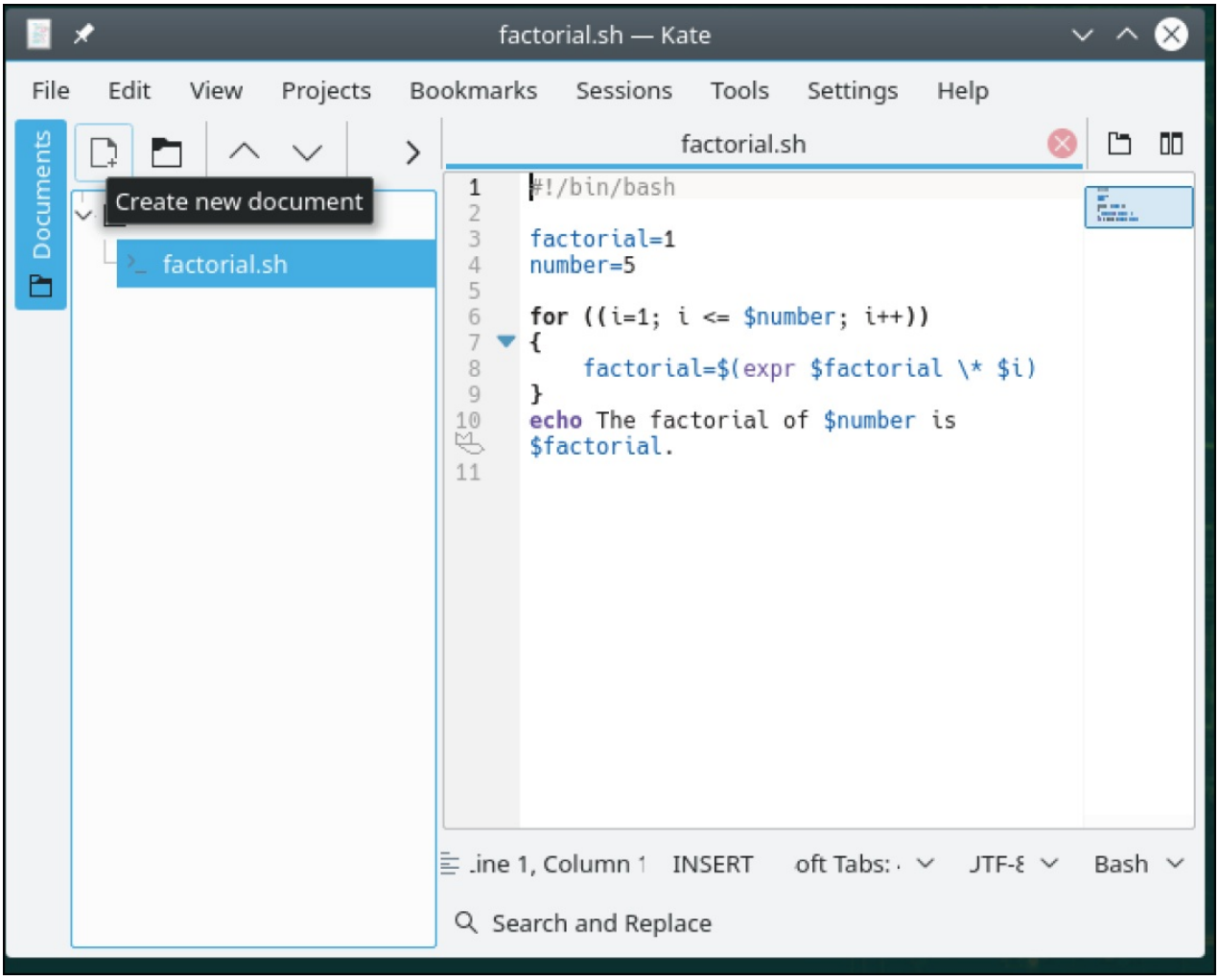

FIGURE 10-11 The Kate Documents List

Kate also supports several external plugin applications, which can be activated in the Plugin Manager window, shown in Figure 10-12. You reach this feature by choosing Settings ➪ Configure ➪ Kate ➪ Plugins. In this window, you can select various plugins to make your shell scripting environment more productive.

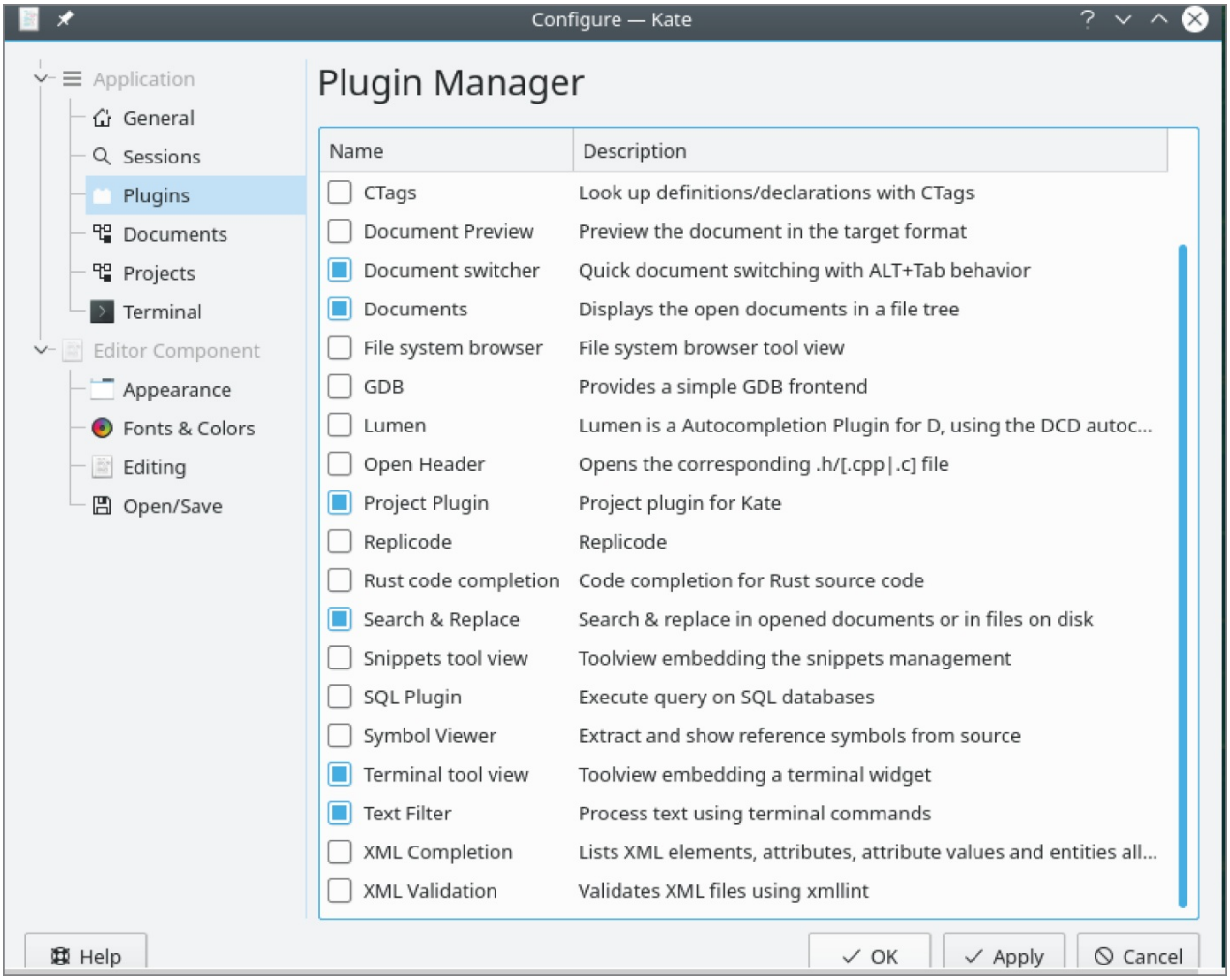

FIGURE 10-12 The Kate Plugin Manager

One great feature of the Kate editor is the built-in terminal plugin (`Terminal tool view`), which provides a terminal window, shown in Figure 10-13. The terminal icon at the bottom text editor's window starts the built-in terminal emulator in Kate (using the KDE Konsole terminal emulator is covered in Chapter 2, “Getting to the Shell”).

> **TIP**
>
> **If you don't see the terminal icon at the bottom of the Kate window, most likely you have not activated the `Terminal tool view` plugin. Once you have reached the Plugin Manager window (described earlier) and have selected the `Terminal tool view` plugin, be sure to click the Apply icon to activate this feature.**

This terminal emulator feature horizontally splits the current editing window, creating a new window with Konsole running in it. You can now enter command-line commands, start programs, or check on system settings without having to leave the editor! To close the terminal window, just type **`exit`** at the command prompt.

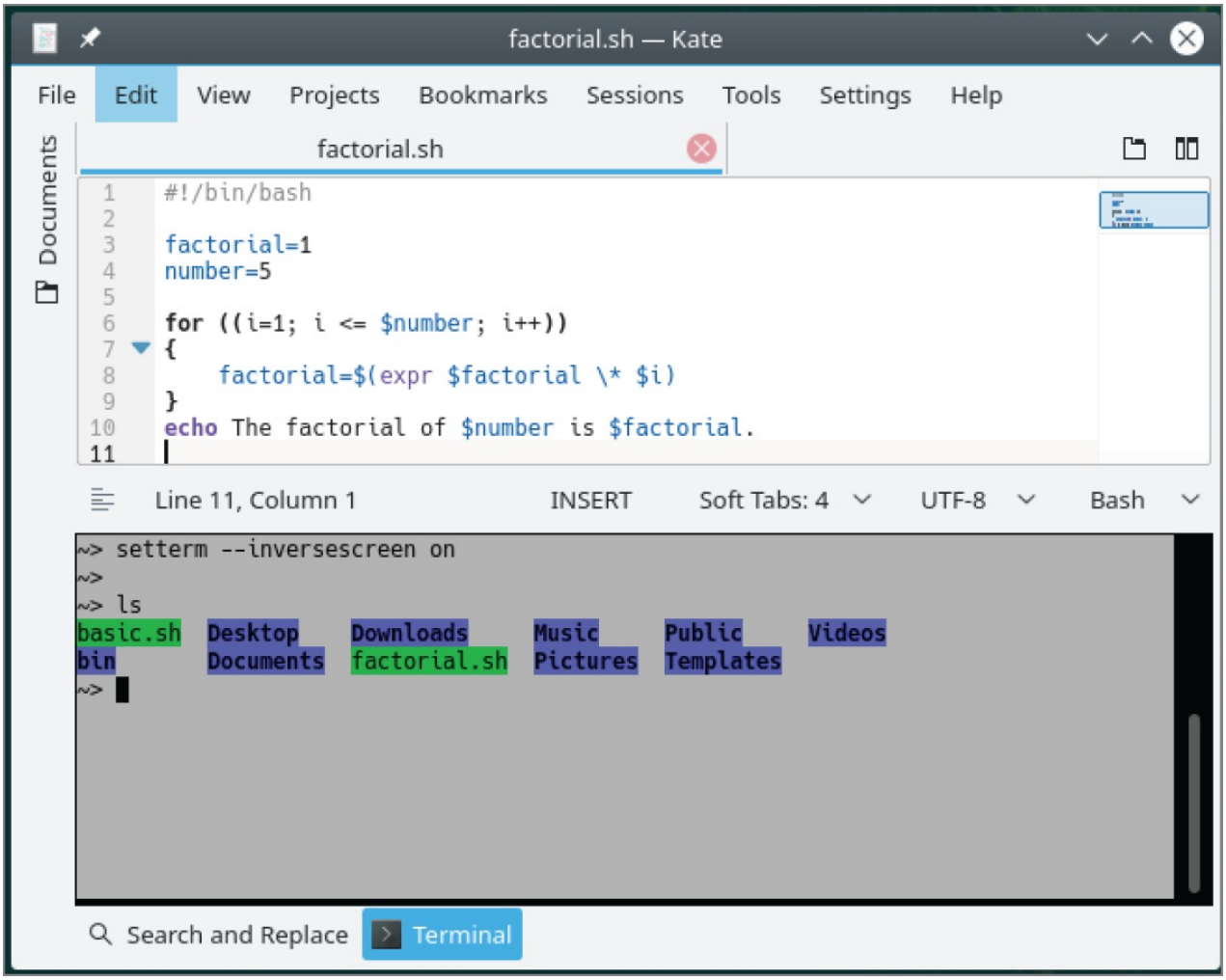

FIGURE 10-13 The Kate built-in terminal window

As you can tell from the terminal feature, Kate also supports multiple windows. The Window menu bar item (View) provides options to perform these tasks, such as the following:

- Create a new Kate window using the current session.
- Split the current window vertically to create a new window.
- Split the current window horizontally to create a new window.
- Close the current window.

The View menu also allows you to control the editor window's functionality such as displaying various tools, changing the font size, and showing nonprintable characters. Kate is rich in features.

NOTE

The Kate editor handles files in sessions. You can have multiple files open in a session, and you can have multiple sessions saved through the Sessions menu. When you start Kate, you can recall saved sessions. This allows you to easily manage files from multiple projects by using separate workspaces for each project.

To set the configuration settings in Kate, select Settings ⇨ Configure Kate. The Configuration dialog box, shown in Figure 10-14, appears. The Application settings area allows you to configure settings for the Kate items, such as controlling sessions (shown in Figure 10-14), the documents list, and the filesystem browser.

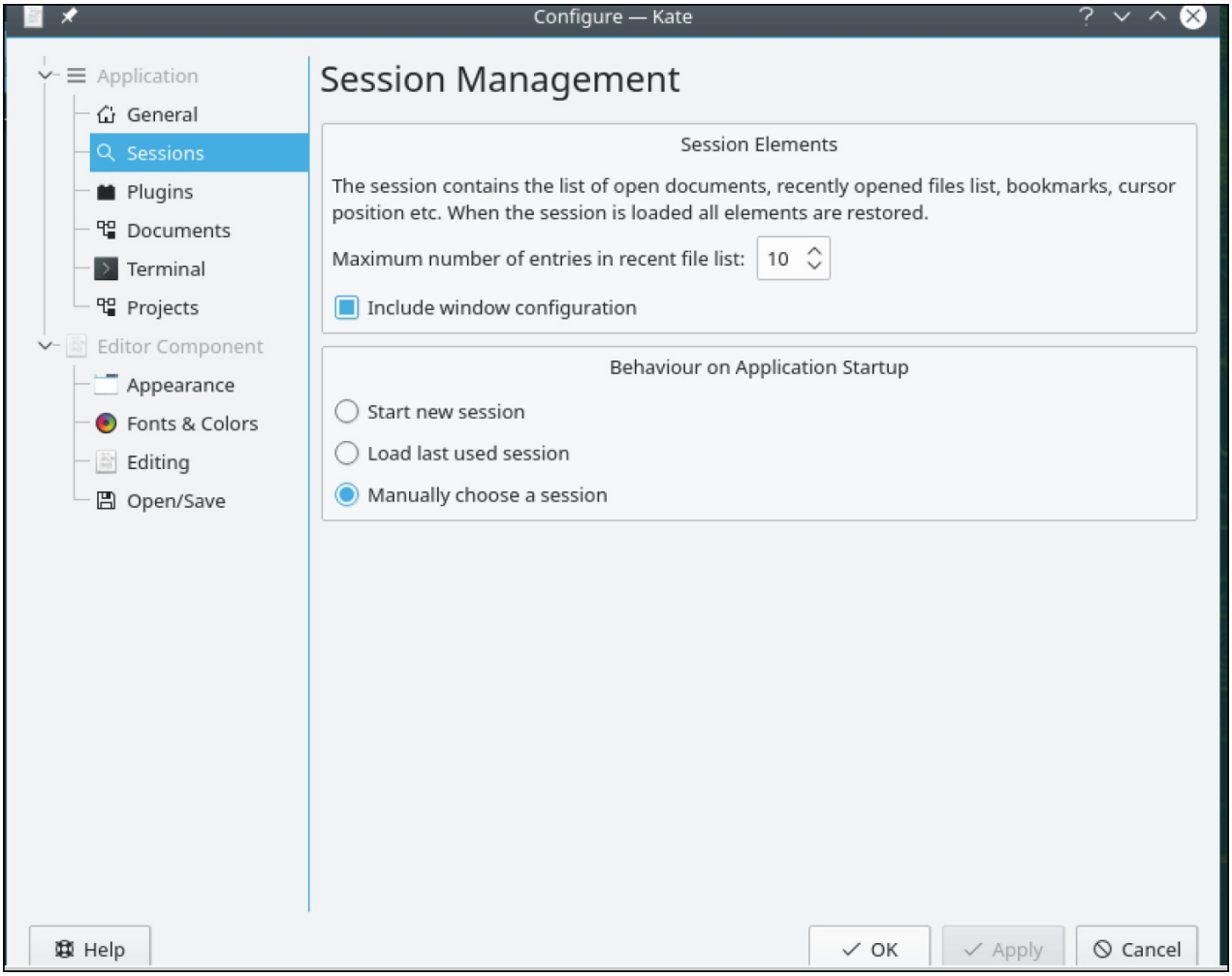

FIGURE 10-14 The Kate configuration dialog box

Kate and Kwrite work well side by side. Kate is a rich MDI editor that acts as an IDE, making it useful for creating and editing shell scripts. KWrite, on the other hand, launches quickly while providing nearly as much power as Kate so that you can make fast fixes to your scripts. Both editors have their place in the world.

Exploring the GNOME Editor

If you're working on a Linux system with the GNOME Shell desktop environment, there's a graphical text editor that you can use as well. The gedit text editor is a basic text editor, with a few advanced features thrown in just for fun. This section walks you through the features of gedit and demonstrates how to use it for your shell script programming.

Starting gedit

In the GNOME Shell desktop environment, accessing gedit is fairly straightforward. Click the Activities icon in the upper-right corner of the desktop window. When the search bar appears, click within the bar to access it, type **gedit** or **text editor**, and then click Text Editor.

> **TIP**
>
> **If gedit is not installed by default on your desktop environment, you can easily install it (see Chapter 9). The package name is `gedit` . You should install the gedit plugins as well, because they provide powerful and advanced features. Their package name, as you might have already guessed, is `gedit-plugins`.**

If desired, you can start gedit from the command-line prompt in a GUI terminal emulator:

```
$ gedit factorial.sh myprog.c
```

When you start gedit with multiple files, it loads all the files into separate buffers and displays each one as a tabbed window within the main editor window, as shown in Figure 10-15.

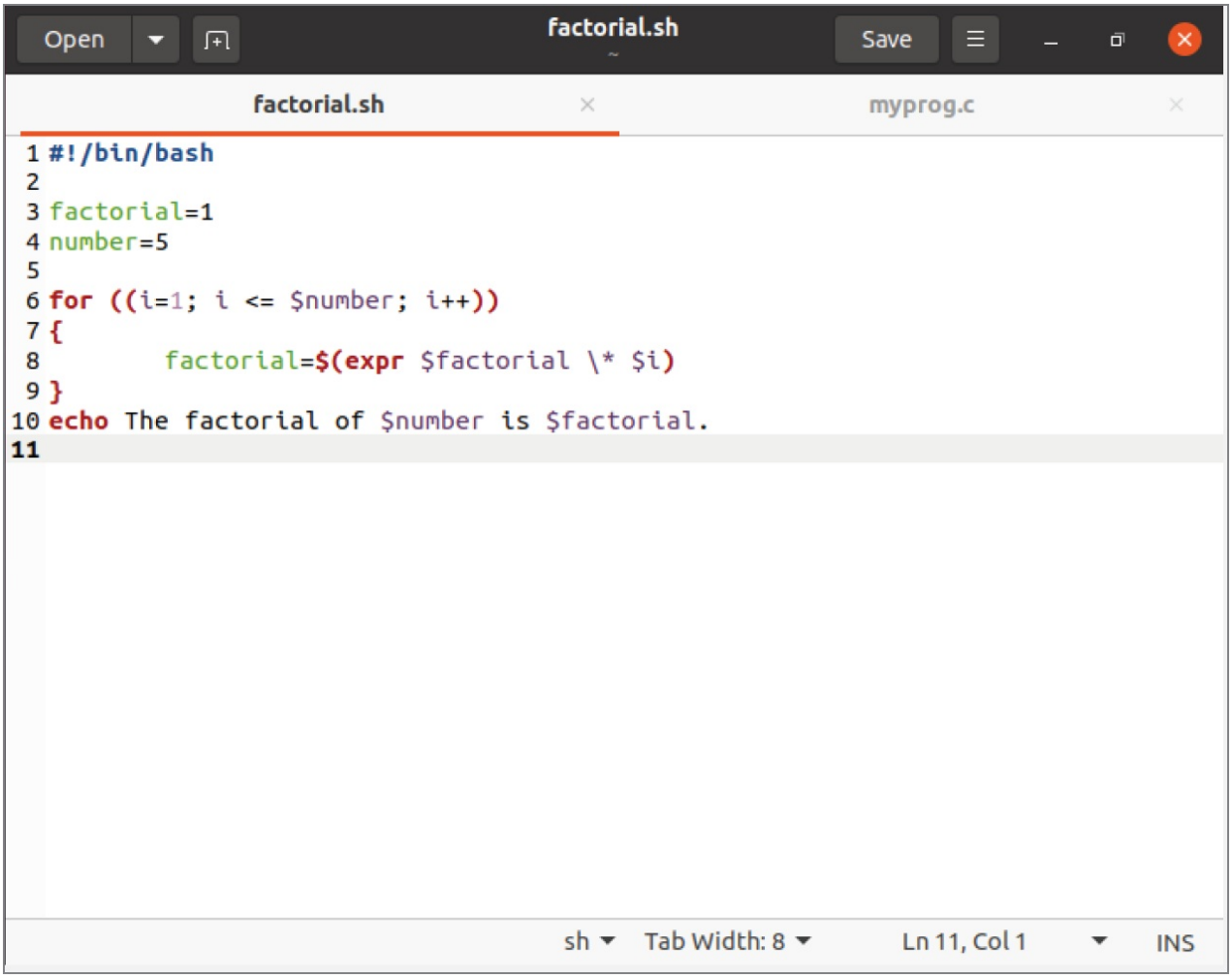

FIGURE 10-15 The gedit main editor window

In Figure 10-15, the left frame in the gedit main editor window shows the documents you're currently editing. The right side shows the tabbed window that contains the buffer text of the second file. If you hover your mouse pointer over each tab, a dialog box appears, showing the full pathname of the file, the MIME type, and the character set encoding it uses.

> **TIP**
>
> **You can quickly jump between the gedit tabs by clicking on the tab. If you prefer shortcut keys, pressing Ctrl+Alt+PageDown will put you into the tab buffer on the right. Pressing Ctrl+Alt+PageUp will take you to the left.**

Understanding basic gedit features

Modern versions of gedit don't use a menu bar. Instead, they use a menu system accessible through a collapsed menu icon (also called a hamburger button) in the title bar that allows you to control files, manage your editing session, configure settings, and so on, as shown in Figure 10-16.

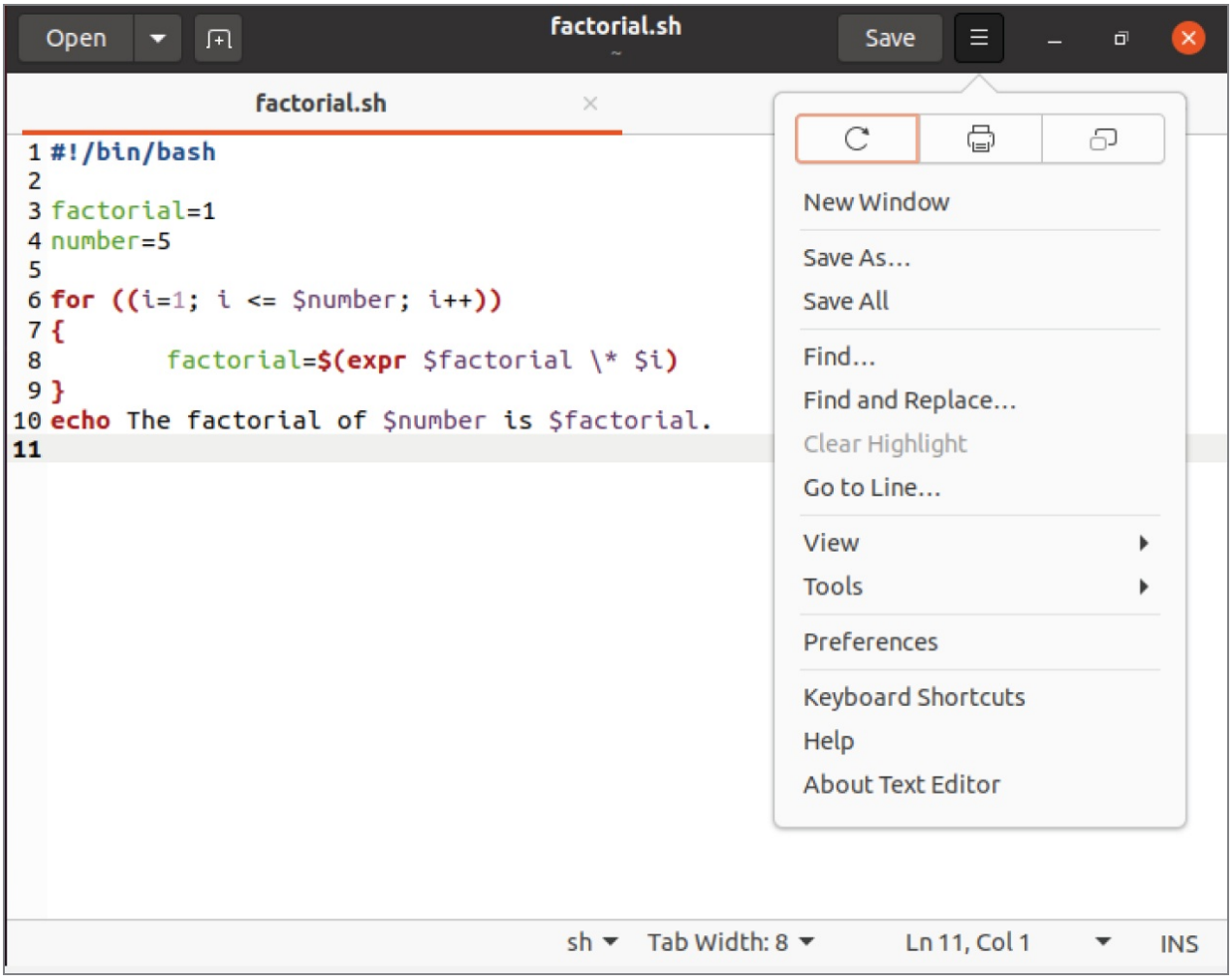

FIGURE 10-16 The gedit menu system

These menu items available are:

- **New Window** opens a new editing window instead of a tab.
- **Save As** saves the current buffered file to a new filename.
- **Save All** saves all the tabs' contents to disk.
- **Find** opens the Find Text dialog box, which allows you to customize a text search, and highlights the found text.
- **Find and Replace** displays the Find/Replace pop-up window, which allows you to customize a text search and replace.
- **Clear Highlight** removes the highlighting of found text.
- **Go to Line** opens the Go to Line dialog box, which moves the cursor to the entered line number of the text.
- **View** opens a submenu allowing the selection of displaying a Documents List/File Manager (Side Panel), Embedded Terminal (Bottom Panel), and syntax highlighting (Highlight Mode).
- **Tools** displays a submenu allowing the activation of a spell checker (Check Spelling), changing the spell checker's language (Set Language), highlights of misspellings (Highlight Misspelled Words), displaying text stats (Document Statistics), and selecting a particular date and time format for insertion (Insert Date And Time)
- **Preferences** opens a pop-up window providing customization of the gedit editor's operation, including such choices as displaying the line numbers, tab stops, text fonts and colors, and activated gedit plugins.
- **Keyboard Shortcuts** displays a brief list of available gedit keyboard shortcuts.
- **Help** provides access to the full gedit manual.
- **About Text Editor** shows information concerning the gedit version, description, website, and so on.

One item not on the menu is a basic save feature that lets you save the text in the current tab buffer with its original filename. That's because gedit conveniently provides a Save icon on the title bar (see Figure 10-16). Just a click on the icon and the file is saved. If you prefer to use the keyboard instead, the Ctrl+S shortcut provides the same function.

NOTE

Your Linux desktop environment may have an older or newer version of gedit than the one shown in these figures. In this case, your gedit text editor could have diverse options or even the same options, but they are available in slightly different menu locations. Consult your distribution's gedit Help menu for more assistance.

The Side Panel in gedit provides functionality similar to the Documents List in Kate. Access this feature by clicking the menu hamburger icon and then choosing View. When the View submenu appears, click the box next to Side Panel. These actions result in something similar to Figure 10-17.

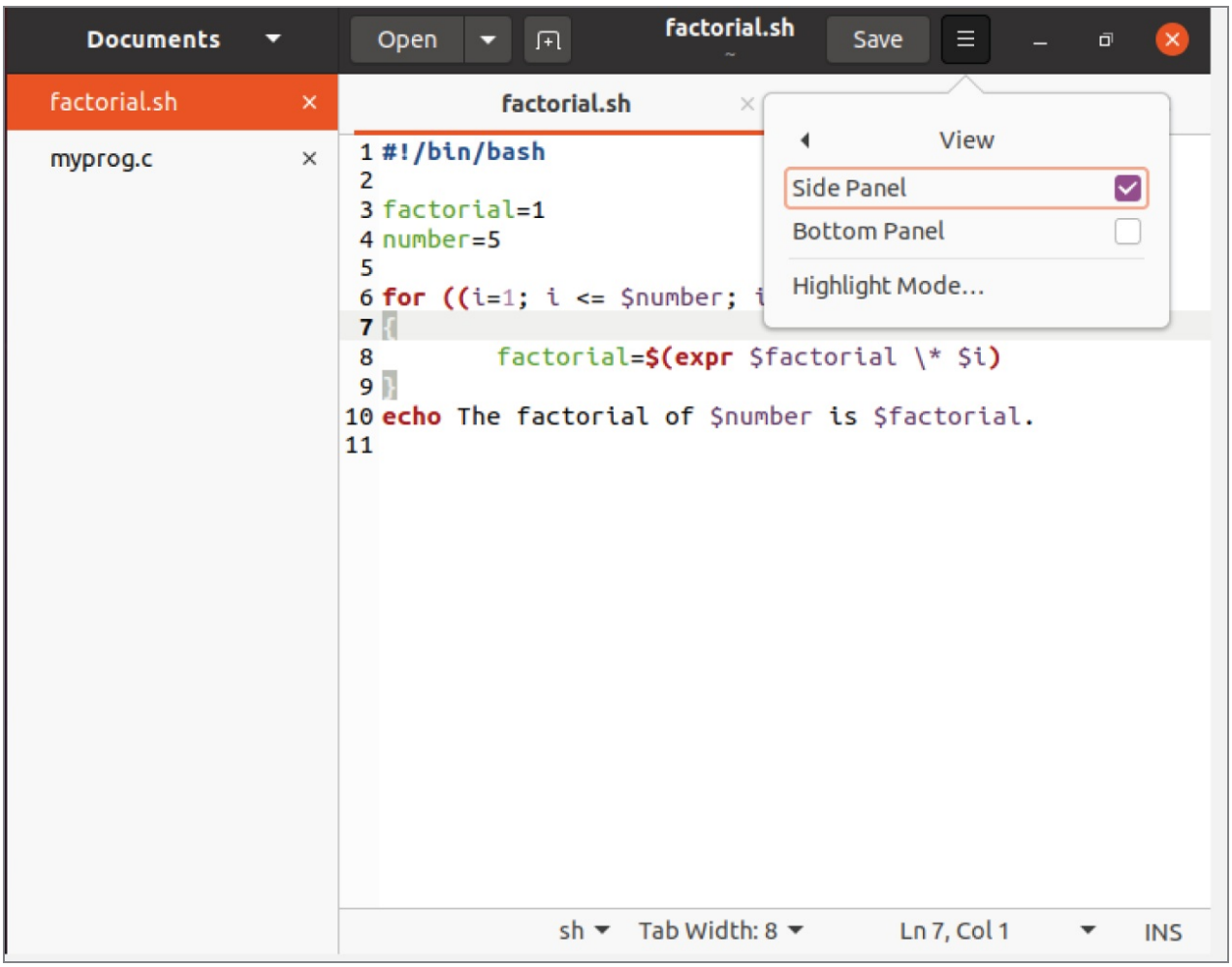

FIGURE 10-17 The gedit Side Panel

The gedit Side Panel provides quick switching between open documents. You can also switch to a file manager by clicking Documents and selecting File Browser, as shown in Figure 10-18.

Within the File Browser pane, you can look through various folders and find other files to edit. If desired, switch back to Documents by clicking File Browser and selecting Documents.

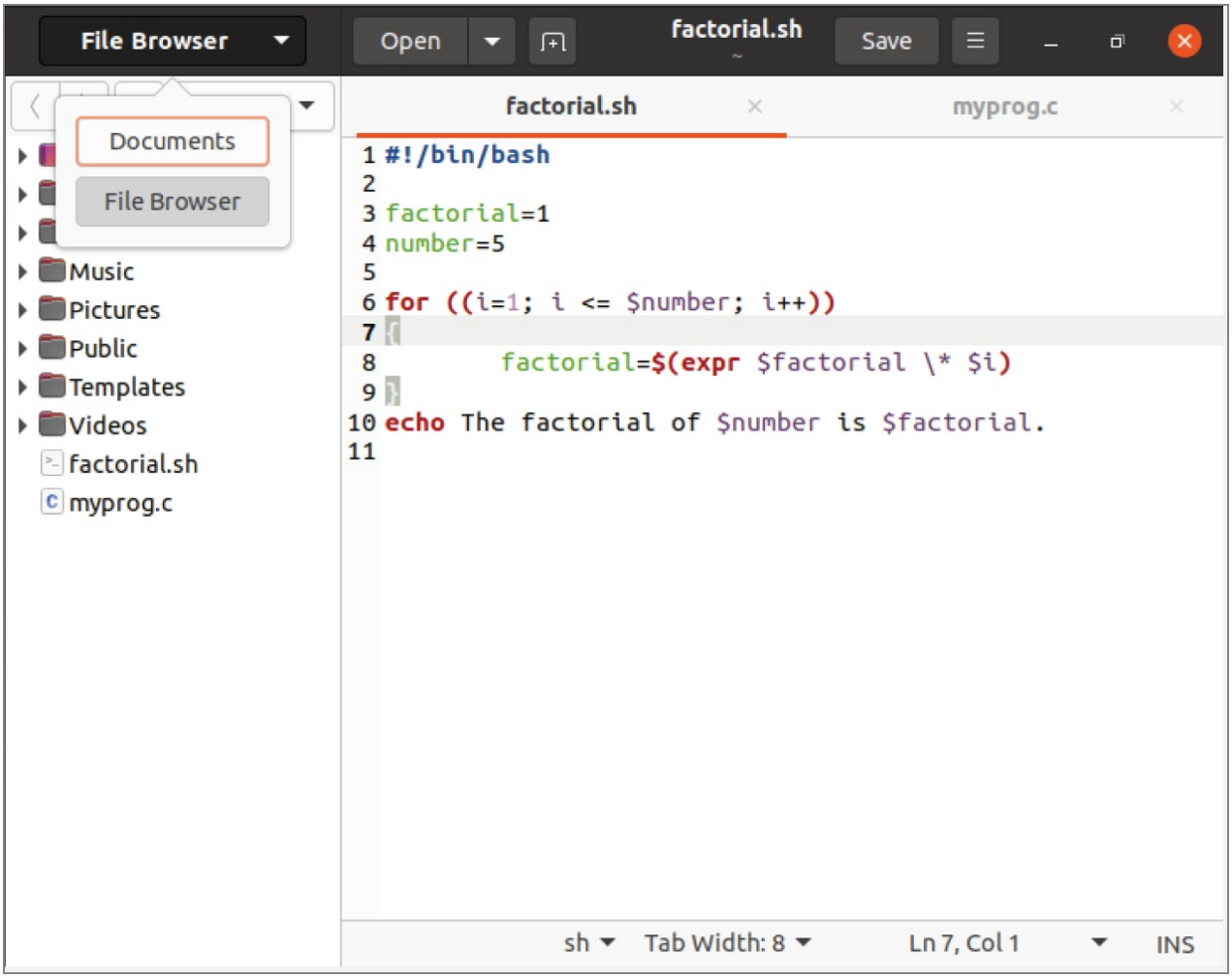

FIGURE 10-18 The gedit Side Panel's file manager

> **TIP**
>
> **Instead of using the mouse and the gedit menu system, quickly open or close the Side Panel by pressing the F9 key.**

The File Browser is a gedit plugin. If for some reason you cannot reach the File Browser from the Side Panel, it may not be enabled or possibly not installed. You can always open files using the Ctrl+O keyboard shortcut, but the File Browser is a useful plugin. We cover more about the managing gedit plugins next.

Managing plugins

The Plugins tab within the gedit Preferences window (shown in Figure 10-19) provides control over the plugins used in gedit. Plugins are separate programs that can interface with gedit to provide additional functionality.

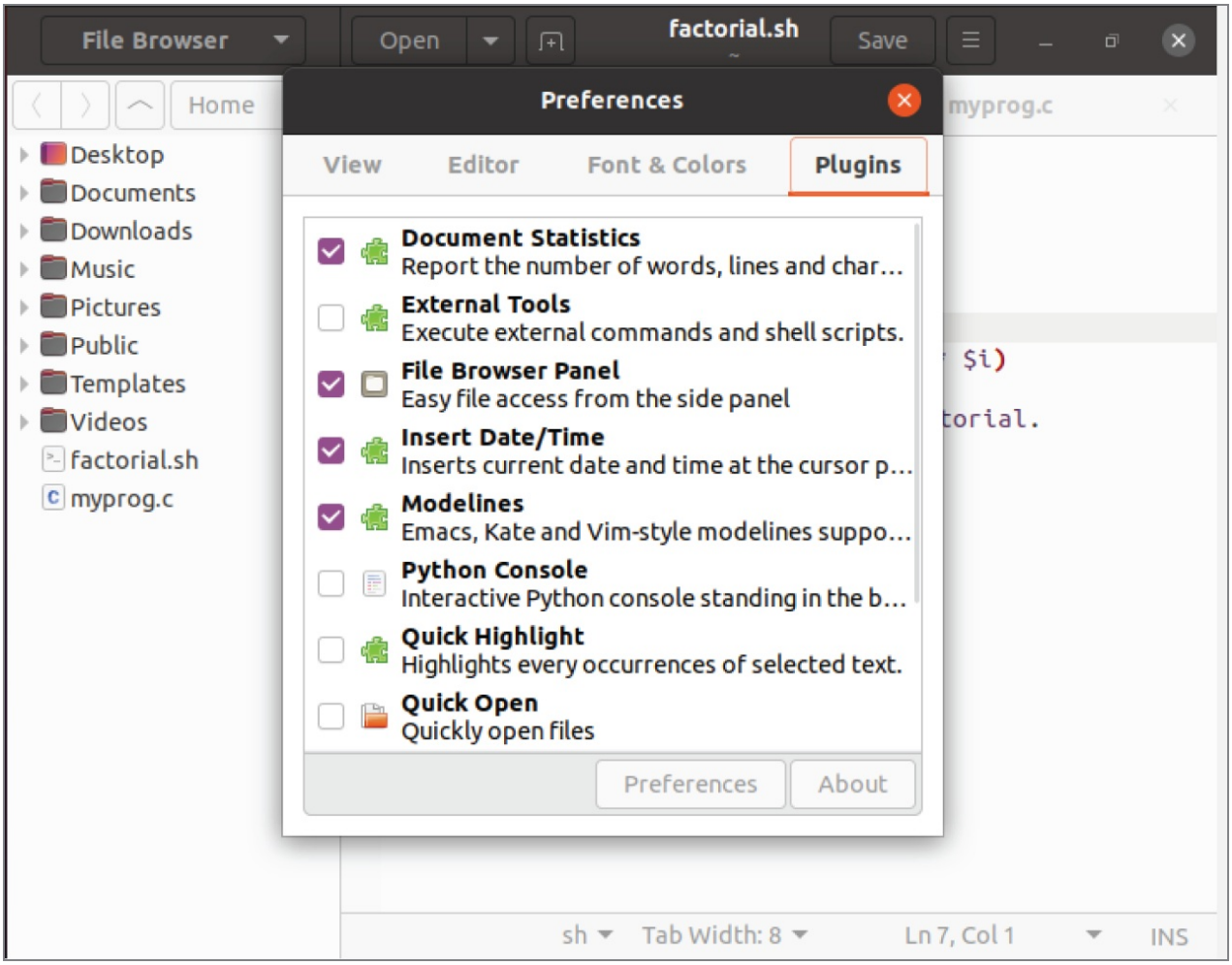

FIGURE 10-19 The gedit Plugins tab

Typically only basic gedit plugins are installed by default. Table 10-5 describes the basic plugins that are currently available in the GNOME desktop's gedit application.

If desired, you can get additional useful features, such as an embedded terminal, by installing the `plugins` metapackage (see Chapter 9), as shown here on Ubuntu:

```
$ sudo apt install gedit-plugins
[sudo] password for christine:
[...]
0 upgraded, 29 newly installed, 0 to remove and 77 not upgraded.
Need to get 2,558 kB of archives.
After this operation, 13.6 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
[...]
Setting up gedit-plugins (3.36.2-1) ...
$
```

Once you have the additional plugins installed on your system, you'll see many additional choices on the Plugins tab within the gedit Preferences window, as shown in Figure 10-20.

TABLE 10-5 The GNOME Desktop gedit Plugins

Plugin	Description
Document Statistics	Reports the number of words, lines, characters, and non-space characters.
External Tools	Provides a shell environment in the editor to execute commands and scripts.
File Browser Panel	Provides a simple file browser to make selecting files for editing easier.
Insert Date/Time	Inserts the current date and time in several formats at the current cursor position.
Modelines	Provides Emacs, Kate, and vim-style message lines at the bottom of the editor window.
Python Console	Provides an interactive console at the bottom of the editor window for entering commands using the Python programming language.
Quick Highlight	Highlights all the matching text of a selection.
Quick Open	Opens files directly in the gedit edit window.
Snippets	Allows you to store often-used pieces of text for easy retrieval anywhere in the text.
Sort	Quickly sorts the entire file or selected text.
Spell Checker	Provides dictionary spell checking for the text file.

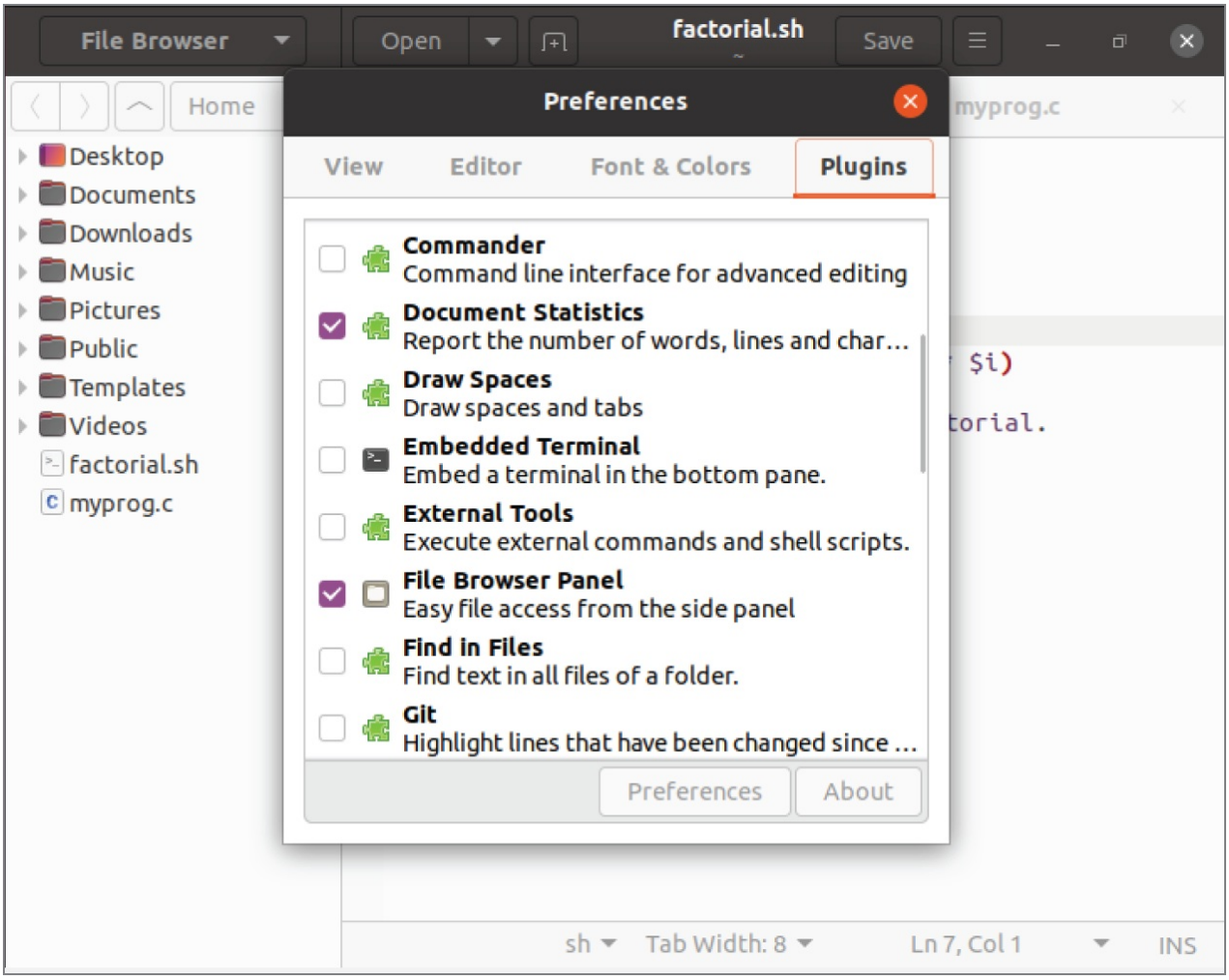

FIGURE 10-20 The gedit Plugins tab after installation

Plugins that are enabled show a check mark in the check box next to their name. Enabling a plugin does not start it. If you want to use, for example, the Embedded Terminal, you must enable it in the Preferences tab first. After that, you can access the plugin by clicking the menu hamburger icon and choosing View, and when the View submenu appears, clicking the box next to Bottom Panel to open the Embedded Terminal, as shown in Figure 10-21.

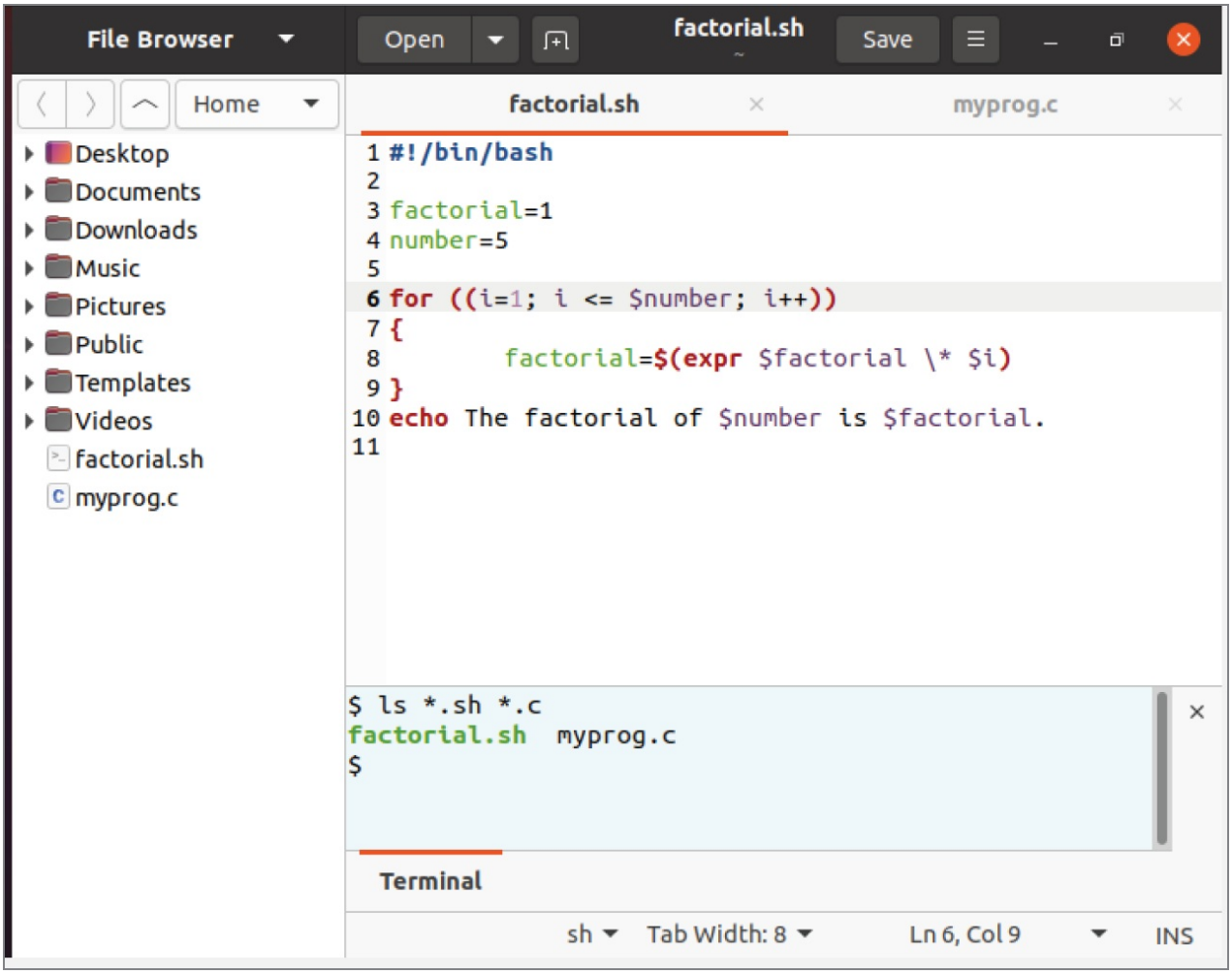

FIGURE 10-21 The gedit Embedded Terminal plugin

This chapter has covered just a few of the text editors available on Linux. If you find that the text editors described here don't meet your needs, you have options. Many more Linux editors are available, such as Geany, Sublime Text, Atom, Visual Studio Code, and Brackets, to name a few. All these editors can help you as you begin your Bash shell script writing journey.

Summary

When it comes to creating shell scripts, you need some type of text editor. Several popular text editors are available for the Linux environment. The most popular editor in the Unix world, vi, has been ported to the Linux world as the vim editor. The vim editor provides simple text editing from the console, using a rudimentary full-screen graphical mode. The vim editor provides many advanced editor features, such as text searching and replacement.

Another editor that has been ported from the Unix world to Linux is the nano text editor. The vim editor can be rather complex, but the nano editor offers simplicity. The nano editor allows quick text editing in console mode.

Another popular Unix editor — Emacs — has also made its way to the Linux world. The Linux version of Emacs has both a console and a graphical mode, making it the bridge between the old world and the new. The Emacs editor provides multiple buffer areas, allowing you to edit multiple files simultaneously.

The KDE Project created two editors for use in the KDE Plasma desktop. The KWrite editor is a simple editor that provides basic text-editing features, along with a few advanced features, such as syntax highlighting for programming code and line numbering. The Kate editor provides more advanced features for programmers. One great feature in Kate is a built-in terminal window. You can open a command-line interface session directly in the Kate editor without having to open a separate terminal emulator window. The Kate editor also allows you to open multiple files, providing different windows for each opened file.

The GNOME Project also provides a graphical text editor for programmers. The gedit editor provides some advanced features such as code syntax highlighting and line numbering, but it was designed to be a bare-bones editor. To spruce up the gedit editor, developers created plugins, which expand the features available in gedit. Plugins include a spell-checker, a terminal emulator, and a file browser.

This wraps up the background chapters on working with the command line in Linux. The next part of the book dives into the shell scripting world. The next chapter starts off by showing you how to create a shell script file and how to run it on your Linux system. It also shows you the basics of shell scripts, allowing you to create simple programs by stringing multiple commands together into a script you can run.